

ARC_Instrument.dll

The ARC_Instrument.dll is provided by Acton Research Corp (ARC) to allow for simplified low-level control of up to 16 Acton Research Corp. Readout Systems/SpectraPro/AM/VM Monochromators/Spectrographs and External Filter Controllers.

The ARC_xxxx.dll family of dll's is designed to provide the customer the level of instrument control he needs. The modular design of the dll's allow the user to upgrade his dll as different types of Acton supplied equipment is added with minimal code change. The user should only use a single ARC_xxxx.dll and not mix ARC_xxxx.dll's in an application. ARC_SpectraPro.dll contains all ARC_FilterWheel.dll functions. ARC_Instrument.dll contains all ARC_SpectraPro.dll functions.

The family is composed of the following dll's.

1. ARC_FilterWheel.dll : For users that only use an Acton External Filter Wheel Controller.
2. ARC_SpectraPro.dll : For users that use Acton SpectraPro/AM/VM Monochromators/Spectrographs and External Filter Controllers.
3. ARC_Instrument.dll : For users that use Acton Readout Systems/SpectraPro/AM/VM Monochromators/Spectrographs and External Filter Controllers.

The DLL is broken into groups of functions.

1. Readout System Communications : These functions are used for setting up communications with the ARC instrument(s) attached.
2. Readout System Information : These functions provided basic information about Open ARC instruments. This information includes such things as; model and serial number.
3. Readout System Detectors : These functions provide are used for reading and controlling single point detectors attached to the readout system(s).
4. NCL Readout Filters : These functions provide control of filter wheels in the NCL Readout System.
5. NCL Readout Shutter : These functions provide control of the two shutters in the NCL Readout System.
6. NCL Readout TTL : These functions provide control of the four input and four output TTL's lines in the NCL Readout System.
7. NCL Readout Mono : These functions provide information on up to two Monochromators that may be connected to a NCL Readout system. The Monochromators are setup and enumerated when an NCL Readout System is opened. Control of the NCL Monochromators is transparently provided through the normal monochromator functions.
8. Monochromator Communications : These functions are used for setting up communications with the ARC instrument(s) attached.
9. Monochromator Information : These functions provided basic information about Open ARC instruments. This information includes such things as; model, serial number, focal length, half angle, detector angle, and if the instrument is a double monochromator.
10. Monochromator Wavelength : These functions set and read the current wavelength (center wavelength for a Spectrograph) of an instrument. Functions are provided for the most common wavelength units, but it should be noted that all ARC instruments internally use nanometers (nm), so all functions call `ARC_get_Mono_Wavelength_nm / ARC_set_Mono_Wavelength_nm` and then convert to the requested units.
11. Monochromator Grating : These functions are used for selecting the current grating and providing information about the grating.
12. Monochromator Diverter Mirrors : These functions provide control over diverter mirror in instruments with multiple entrance or exit ports. They are not valid for instruments not equipped with these options. Note these functions do not work with all older SD-748/SP-275/SP-500(pre SP-500i)/SP-750 (pre SP-750i) controllers.
13. Monochromator Slits : These functions provide control over motorized slits (both continuously variable and index able). They are not valid for instruments not equipped with these options. Note these functions do not work with all older SD-748/SP-275/SP-500(pre SP-500i)/SP-750 (pre SP-750i) controllers.
14. Monochromator Advanced Functions : These are advanced functions that most normal users of the DLL will not need. They allow for debugging of hardware and installing/adjusting gratings.
15. Monochromator Filters : These functions provide control of filter wheels in instruments that contain and integrated filter wheel controller. At present this is not a standard feature and the functions are invalid for most instruments. This function does not provide control of a filter wheel controller external to the monochromator.
16. Monochromator Shutter : These functions provide control of the optional shutter for the SpectraPro SP-2xxx and SD-3 controllers.
17. Monochromator Setup : These functions are used internally by Acton Research Corp. to setup SpectraPro SP-2xxx and SD-3 controllers. These functions are provided for completeness. Documentation and the access to the functions is withheld by Acton Research Corp.
18. Functions for the controlling External Filter Wheel Controllers.
 1. Communications : These functions are used for setting up communications with the ARC instrument(s) attached.
 2. Information : These functions provided basic information about Open ARC instruments. This information includes such things as; model and serial number. (Note, older filter controllers do not store the serial number electronically).
 3. Filters : These functions provide control of the external ARC Filter controllers.

Working with the DLL is a multi step process.

1. First identify the number of monochromators / spectrographs attached by calling `ARC_Search_For_Mono`. Each instrument enumerates starting with zero up to the number of instruments found minus one.
`ARC_get_Mono_preOpen_Model` will return the model string in a variant for the enumerated value passed to it.
2. `ARC_Open_Mono` is then called to open the enumeration. The function will return false (result = 0) if it failed to open the enumeration.
3. The user then calls the functions he needs to use to operate the instrument. Once opened, a monochromator port can be left open until the software exits.
4. When the user is done with the instrument (usually program exit) the user calls `ARC_Close_Mono` to close the port.

Data types used :

Most functions return a 16 bit unsigned integer result. True is defined as not equal to zero. The function failed if true is not returned.

In VB this is an integer

In VCC this is a short

In Delphi this is a WordBool

Parameters passed to and from a function are of the following types ...

Integers values are 32 bit integers

In VB this is a long

In VCC this is a long

In Delphi this is an integer

Floating-point values are doubles. This is true for most languages.

Strings are wrapped inside of the variant data type. This allows compatibility with most programming languages

In VB this is a variant

In VCC this is a variant

In Delphi this is an OLEVaraint (OLEVariant does not work the same as variant, you must include variants in the uses clause on Delphi 6 and higher)

Boolean values are 16 bit integers defined as true if not equal to zero.

In VB this is an integer

In VCC this is a short

In Delphi this is a wordbool

Functions Provided :

Readout System Communications - Readout Systems/SpectraPro/AM/VM Monochromators/Spectrographs and External Filter Controllers are opened in one of two ways, using `ARC_Search_For_Inst` / `ARC_Open_Readout` / `ARC_Open_Mono` / `ARC_Open_Filter` or to directly open the COM port with `ARC_Open_Readout_Port` / `ARC_Open_Mono_Port` / `ARC_Open_Filter_Port`.

ARC_Search_For_Inst , function used to search for all attached Acton Research Instruments. With the exception of `ARC_Ver`, must be the first function called.

ARC_get_Num_Found_Inst_Ports , Returns the value last returned by **ARC_Search_For_Inst**. Can be called multiple times.

ARC_Open_Readout , function to open a Readout System (NCL/NCL-Lite)

ARC_Open_Readout_Port , function to open a Readout System (NCL/NCL-Lite) by addressing a specific COM port.

ARC_Close_Enum , function to close an open enumeration.

ARC_Valid_Enum , function returns if an enumeration is both valid and the instrument is open.

ARC_Valid_Readout_Enum , function returns if a Readout System (NCL/NCL-Lite) enumeration is both valid and the instrument is open.

ARC_get_Readout_preOpen_Model , returns the model string of an Readout System (NCL/NCL-Lite) not yet opened. Note, `ARC_Search_For_Inst` needs to be called prior to this call.

direct communications function

ARC_Send_CMD_To_Inst , this is an internal function that provides direct communications with the instrument. Usage of this function should be avoided.

ARC_Ver , this function provides the DLL version. It is the only function that can be called at any time

Readout System Information

ARC_get_ReadOut_Model , returns the model string from the instrument

ARC_get_ReadOut_Serial , returns the serial number from the instrument

Readout System Detectors

Detector Type 1 ... Current

Detector Type 2 ... Voltage

Detector Type 3 ... Photon Counting

Detector Range 0 ... 1x

Detector Range 1 ... 2x

Detector Range 2 ... 4x

Detector Range 3 ... 50x

Detector Range 4 ... 200x

ARC_get_Num_Det, return the number of single point detectors in the Readout System

ARC_Valid_Det_Num, return if the detector number is valid on the Readout System

ARC_get_ReadOut_ITime_ms, return the integration time used for taking a reading

ARC_set_ReadOut_ITime_ms, set the integration time used for taking a reading

ARC_get_Det_Type, return the detector readout mode (Current,Voltage,Photon Counting)

ARC_get_Det_Type_Str, return a variant string of the detector readout mode

ARC_set_Det_Type, set the detector readout type

ARC_set_Det_Photon, set the detector readout type to photon counter

ARC_set_Det_Voltage, set the detector readout type to voltage

ARC_set_Det_Current, set the detector readout type to current

ARC_get_Det_BiPolar, return if a detector takes bipolar (+/-) readings

ARC_get_Det_BiPolar_Str, return a variant string of the detector polarity

ARC_set_Det_BiPolar, set the detector readout to bipolar (+/-)

ARC_set_Det_UniPolar, set the detector readout to unipolar (-)

ARC_get_Det_Range, return the detector range factor

ARC_get_Det_Range_Factor, return the detector range multiplier

ARC_set_Det_Range, set the detector range factor

ARC_get_Det_HVOn, return if the high voltage for a detector is turned on

ARC_set_Det_HV_on, set the detector high voltage to on

ARC_set_Det_HV_off, set the detector high voltage to off

ARC_get_Det_HV_Volts, return the high voltage volts setting

ARC_set_Det_HV_Volts, set the voltage value of the high voltage

ARC_get_Det_NumAvgRead, return the number of readings that are averaged

ARC_set_Det_NumAvgRead, set the number of readings to average

ARC_Det_Read, readout a single detector

ARC_Det_ReadAll, readout all detectors

Readout System Non Blocking Reads

ARC_Det_Start_NonBlock_Read , start to readout a single detector, but return immediately

ARC_Det_NonBlock_Read_Done , determine is a single detector readout has completed and return value

NCL Readout Filters

ARC_get_NCL_Filter_Present, returns if the instrument has an integrated filter wheel setup

ARC_set_NCL_Filter_Present, set the instrument filter wheel to active (NCL only)

ARC_get_NCL_Filter_Position, return the current filter position

ARC_set_NCL_Filter_Position, set the current filter position

ARC_get_NCL_Filter_Min_Pos, returns the min. filter position

ARC_get_NCL_Filter_Max_Pos, returns the max. filter position

ARC_NCL_Filter_Home, home the filter wheel

NCL Readout Shutter

ARC_get_NCL_Shutter_Valid, return if a NCL Readout Shutter number is valid.

ARC_get_NCL_Shutter_Open, return if a NCL Readout shutter is open

ARC_set_NCL_Shutter_Open, set a NCL Readout shutter to a closed state

ARC_set_NCL_Shutter_Closed, set a NCL Readout shutter to a closed state

SpectraHub Shutter Functions

ARC_get_SpectraHub_Shutter_Enabled, return if a SpectraHub Shutter number is enabled

ARC_set_SpectraHub_Shutter_Enable, enable the shutter port on a SpectraHub

ARC_set_SpectraHub_Shutter_Disable, disable the shutter port on a SpectraHub

ARC_get_SpectraHub_Shutter_Open_High, return if the shutter port on a SpectraHub opens on TTL high

ARC_set_SpectraHub_Shutter_Open_High, sets the SpectraHub shutter port to use TTL high to open

ARC_set_SpectraHub_Shutter_Open_Low, sets the SpectraHub shutter port to use TTL low to open

ARC_get_SpectraHub_Shutter_Open, return of the SpectraHub shutter is open

ARC_set_SpectraHub_Shutter_Open, Open the SpectraHub shutter

ARC_set_SpectraHub_Shutter_Closed, Close the SpectraHub shutter

NCL Readout TTL

ARC_get_NCL_TTL_Valid, return if a TTL line is a valid line

ARC_get_NCL_TTL_In, return if an input TTL line is being pulled on by an outside connection

ARC_get_NCL_TTL_Out, return if an output TTL line is on

ARC_set_NCL_TTL_Out_On, turn a TTL line on

ARC_set_NCL_TTL_Out_Off, turn a TTL line off

SpectraHub Trigger functions

ARC_get_SpectraHub_Trig_On, return if reading on a SpectraHub wait for a trigger before starting

ARC_set_SpectraHub_Trig_On, set SpectraHub readings to wait for a trigger before starting

ARC_set_SpectraHub_Trig_Off, turn off SpectraHub readings waiting for a trigger before starting

NCL Readout Mono

ARC_get_NCL_Mono_Setup, return if the open NCL port has a Monochromator attached

ARC_get_NCL_Mono_Enum, return the enumeration of the attached Monochromator

Monochromator Communications

ARC_Open_Mono, function to open a monochromator

ARC_Open_Mono_Port, function to open a monochromator by addressing a specific COM port.

ARC_Valid_Mono_Enum, function returns is a monochromator enumeration is both valid and the instrument is open.

ARC_get_Mono_preOpen_Model, returns the model string of an instrument not yet opened. Note, **ARC_Search_For_Mono** needs to be called prior to this call.

Monochromator Information

ARC_get_Mono_Model, returns the model string from the instrument

ARC_get_Mono_Serial, returns the serial number from the instrument

ARC_get_Mono_Focallength, returns the default focal length of the instrument

ARC_get_Mono_HalfAngle, returns the default half angle of the instrument in radians

ARC_get_Mono_DetectorAngle, returns the default detector angle of the instrument in radians

ARC_get_Mono_Double, returns if the instrument is a double monochromator

ARC_get_Mono_Double_Subtractive, returns if a double monochromator is subtractive instead of additive

ARC_get_Mono_Precision, returns the nm decimal precision of the wavelength drive. Note this is independent of the wavelength step resolution whose coarseness is defined by the grating being used.

ARC_get_Mono_Backlash_Steps, returns the number of backlash steps used when reversing the wavelength drive

ARC_get_Mono_Gear_Steps, returns the number of steps in a set of sine drive gears

ARC_get_Mono_nmRev_Ratio, returns the number of stepper steps per rev of the wavelength drive motor

CCD Pixel Mapping

ARC_get_Mono_Pixel_Map_nm , returns the current pixel wavelength map in nm

ARC_get_Mono_Pixel_Map_ang , returns the current pixel wavelength map in Angstroms

ARC_get_Mono_Pixel_Map_eV , returns the current pixel wavelength map in electron Volts

ARC_get_Mono_Pixel_Map_micron , returns the current pixel wavelength map in microns

ARC_get_Mono_Pixel_Map_absCM , returns the current pixel wavelength map in absolute wavenumber

ARC_get_Mono_Pixel_Map_relCM , returns the current pixel wavelength map in relative wavenumber

Monochromator Wavelength

ARC_get_Mono_Wavelength_nm , returns the current center wavelength of instrument in nm.

ARC_set_Mono_Wavelength_nm , sets the center wavelength of the instrument in nm.

ARC_get_Mono_Wavelength_ang , returns the current center wavelength of instrument in Angstroms. Converted from nm.

ARC_set_Mono_Wavelength_ang , sets the center wavelength of the instrument in angstroms. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_eV , returns the current center wavelength of instrument in electron Volts. Converted from nm.

ARC_set_Mono_Wavelength_eV , sets the center wavelength of the instrument in electron Volts. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_micron , returns the current center wavelength of instrument in microns. Converted from nm.

ARC_set_Mono_Wavelength_micron , sets the center wavelength of the instrument in microns. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_absCM , returns the current center wavelength of instrument in absolute wave number. Converted from nm.

ARC_set_Mono_Wavelength_absCM , sets the center wavelength of the instrument in absolute wavenumbers. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_relCM , returns the current center wavelength of instrument in relative wavenumber based upon the center line value passed to the function in nm. Converted from nm.

ARC_set_Mono_Wavelength_relCM , sets the center wavelength of the instrument in relative wave numbers centered on the wavelength passed in nm. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_Cutoff_nm , returns in nm the max wavelength achievable by the instrument using the current grating. Note, value might be higher than physically achievable on linear drives using the older SD-748 based controllers.

ARC_get_Mono_Wavelength_Min_nm , returns in nm the min wavelength achievable by the instrument using the current grating. Note, value might be lower than physically achievable on linear drives using the older SD-748 based controllers.

Monochromator Grating

ARC_get_Mono_Turret , returns the current grating turret

ARC_set_Mono_Turret , sets the current grating turret

ARC_get_Mono_Turret_Gratings , returns the number of gratings per turret

ARC_get_Mono_Turret_Max , return the number of turrets installed

ARC_get_Mono_Grating , returns the current grating

ARC_set_Mono_Grating , sets the current grating

ARC_get_Mono_Grating_Blaze , returns the blaze of a given grating

ARC_get_Mono_Grating_Density , returns the groove density (grooves per millimeter) of a given grating

ARC_get_Mono_Grating_Installed , returns if a grating is installed

ARC_get_Mono_Grating_Max , returns the max grating position installed (this is usually the number of gratings installed)

Monochromator Mirrors

Mirror Position 1 ... motorized entrance diverter mirror

Mirror Position 2 ... motorized exit diverter mirror

Mirror Position 3 ... motorized entrance diverter on a double slave unit

Mirror Position 4 ... motorized exit diverter on a double slave unit

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

ARC_get_Mono_Diverter_Valid , returns if a motorized diverter position is valid for an instrument

ARC_get_Mono_Diverter_Pos , returns the slit the mirror is pointing to

ARC_set_Mono_Diverter_Pos , sets the port that the unit is to point to

ARC_get_Mono_Diverter_Pos_Var , returns a string describing the port the mirror is pointing to

Monochromator Slits

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Slit Type 0 ... No slit

Slit Type 1 ... Manual Slit

Slit Type 2 ... Fixxed Slit Width

Slit Type 3 ... Focal Plane adapter

Slit Type 4 ... Continuous Motor

Slit Type 5 ... Fiber Optic

Slit Type 6 ... Index able Slit

Slit Type 9 ... Special Continuous Motor

ARC_get_Mono_Slit_Type , returns the slit type for a given slit

ARC_get_Mono_Slit_Type_Var , returns a string descriptor of a given slit

ARC_get_Mono_Slit_Width , returns the slit width of a motorized slit (Types 4,6, and 9)

ARC_set_Mono_Slit_Width , sets the slit width of a motorized slit (Types 4,6, and 9)

ARC_Mono_Slit_Home , homes a motorized slit (Types 4,6, and 9)

ARC_Mono_Slit_Name_Var , returns a text descriptor of a slit position

ARC_Calc_Mono_Slit_BandPass , calculates and returns the band pass for a provided slit width

ARC_get_Mono_Slit_BandPass , returns the band pass of a motorized slit (Types 4,6, and 9)

ARC_set_Mono_Slit_BandPass , sets a motorized slit width of a motorized slit (Types 4,6, and 9)

Special Double Function

ARC_get_Mono_Double_Intermediate_Slit , If a monochromator is a double return the intermediate slit position

ARC_get_Mono_Exit_Slit , Return the exit slit position for a monochromator. Function works with single and double monochromators.

Monochromator Filter

ARC_get_Mono_Filter_Present , returns if the instrument has an integrated filter wheel
ARC_get_Mono_Filter_Position , returns the current filter position
ARC_set_Mono_Filter_Position , sets the current filter position
ARC_get_Mono_Filter_Min_Pos , returns the min. filter position
ARC_get_Mono_Filter_Max_Pos , returns the max. filter position
ARC_Mono_Filter_Home , homes the filter wheel

Monochromator Shutter Functions

ARC_get_Mono_Shutter_Valid, returns if the instrument has an integrated shutter
ARC_get_Mono_Shutter_Open, returns if the integrated shutter is open
ARC_set_Mono_Shutter_Open, sets the integrated shutter position to open
ARC_set_Mono_Shutter_Closed, sets the integrated shutter position to closed

Monochromator Advanced Functions

Gear

ARC_get_Mono_Int_Led_On , returns if the interrupter LED is on
ARC_set_Mono_Int_Led , turns on and off the interrupter LED
ARC_get_Mono_Motor_Int , read the motor gear interrupter
ARC_get_Mono_Wheel_Int , read the wheel gear interrupter
ARC_Mono_Move_Steps , move the grating a set number of steps
ARC_get_Mono_Sine_Drive, returns if the gearing system has sine instead of a linear drive system.

Monochromator Grating Values

ARC_get_Mono_Init_Grating , returns the initial grating (on instrument reboot)
ARC_set_Mono_Init_Grating , sets the initial grating (on instrument reboot)
ARC_get_Mono_Init_Wave_nm , returns the initial wavelength (on instrument reboot)
ARC_set_Mono_Init_Wave_nm , sets the initial wavelength (on instrument reboot)
ARC_get_Mono_Init_ScanRate_nm , returns the initial wavelength scan rate (on instrument reboot)
ARC_set_Mono_Init_ScanRate_nm , sets the initial wavelength scan rate (on instrument reboot)
ARC_get_Mono_Grating_Offset , returns the offset if a grating
ARC_get_Mono_Grating_Gadjust , returns the GAdjust of a grating
ARC_set_Mono_Grating_Offset , sets the grating offset
ARC_set_Mono_Grating_Gadjust , sets the grating GAdjust
ARC_Mono_Grating_Calc_Offset , calculate a new grating offset
ARC_Mono_Grating_Calc_Gadjust , calculate a new grating GAdjust
ARC_Mono_Grating_UnInstall , Uninstall a grating
ARC_Mono_Grating_Install , Install a grating
ARC_Mono_Reset , Reboot an instrument
ARC_Mono_Restore_Factory_Settings , restore the instrument factory settings
ARC_Mono_Save_Factory_Settings, reserved

Monochromator Scan Values

ARC_get_Mono_Scan_Rate_nm_min , return the current wavelength scan rate
ARC_set_Mono_Scan_Rate_nm_min , set the wavelength scan rate
ARC_Mono_Start_Scan_To_nm , start a wavelength scan
ARC_Mono_Scan_Done , check if a scan has completed
ARC_Mono_Start_Jog , start jogging the wavelength of an instrument
ARC_Mono_Stop_Jog , end a wavelength jog

Monochromator SP-2xxx and SD-3 Reserved Setup functions

ARC_Mono_Set_Model, reserved
ARC_Mono_Set_Serial, reserved
ARC_Mono_Get_Board_Serial, reserved
ARC_Mono_Set_Board_Serial, reserved
ARC_Mono_Set_Drive_Linear, reserved
ARC_Mono_Set_Drive_Sine, reserved
ARC_Mono_Set_Gratings_Per_Turret , reserved
ARC_Mono_Set_Halfangle, reserved
ARC_Mono_Set_Gear_Ratio, reserved
ARC_Mono_Set_Backlash, reserved
ARC_Mono_Read_Motors, reserved
ARC_Mono_Motor_set_Use, reserved
ARC_Mono_Motor_set_Stop, reserved
ARC_Mono_Motor_set_Speed, reserved
ARC_Mono_Motor_set_StepRev, reserved
ARC_Mono_Motor_set_eoptions, reserved
ARC_Mono_Grating_Install_NoHello, reserved
ARC_Mono_Grating_UnInstall_NoHello, reserved
ARC_Mono_Motor_set_Motor_Pos_Step, reserved
ARC_Mono_Motor_set_Motor_Min_Pos, reserved
ARC_Mono_Motor_set_Motor_Max_Pos, reserved
ARC_Mono_Motor_set_SlitClamp, reserved
ARC_Mono_Motor_set_SFreq, reserved
ARC_Mono_Motor_set_Microns_to_Steps, reserved
ARC_Mono_Motor_set_Steps_to_Microns, reserved
ARC_Mono_Motor_set_SlitHome, reserved
ARC_Mono_Motor_get_Slit_Hz, reserved
ARC_Mono_Motor_get_Motor_Chan, reserved
ARC_Mono_Motor_get_Motor_StepStop, reserved
ARC_Mono_Motor_get_Motor_Accel, reserved
ARC_Mono_Motor_get_Motor_lraf, reserved
ARC_Mono_Motor_get_Motor_hraf, reserved
ARC_Mono_Motor_get_Motor_mper, reserved
ARC_Mono_Motor_get_Motor_App, reserved
ARC_Mono_Motor_get_Motor_Index_Min_Pos , reserved
ARC_Mono_Motor_get_Motor_Index_Max_Pos, reserved
ARC_Mono_Motor_get_Motor_Speed, reserved
ARC_Mono_Motor_get_Motor_Step_Offset, reserved
ARC_Mono_Motor_get_Motor_Step_Rev, reserved
ARC_Mono_Motor_get_Motor_Step_Pos, reserved
ARC_Mono_Set_Focallength, reserved
ARC_Mono_Set_DetectorAngle, reserved
ARC_Mono_Clear_Mono_Setup, reserved

External Filter Communications

ARC_Open_Filter , function to open a filter controller.

ARC_Open_Filter_Port , function to open a filter controller by addressing a specific COM port.

ARC_Valid_Filter_Enum , function returns if a filter enumeration is both valid and the instrument is open.

ARC_get_Filter_preOpen_Model , returns the model string of an instrument not yet opened. Note,

ARC_Search_For_Inst needs to be called prior to this call.

direct communications function

ARC_Send_CMD_To_Filter , this is an internal function that provides direct communications with the instrument.

Usage of this function should be avoided.

External Filter Information

ARC_get_Filter_Model , returns the model string from the instrument

ARC_get_Filter_Serial , returns the serial number from the instrument

External Filter

ARC_get_Filter_Present , returns if the instrument has an integrated filter wheel

ARC_get_Filter_Position , returns the current filter position

ARC_set_Filter_Position , sets the current filter position

ARC_get_Filter_Min_Pos , returns the min. filter position

ARC_get_Filter_Max_Pos , returns the max. filter position

ARC_Filter_Home , homes the filter wheel

Pre Open Model Types as Integers

Model Type -1 ... Unknown, not supported
Model Type 0 ... Unknown, not supported
Model Type 1 ... GIMS (reserved for, none currently supported by dll)
Model Type 2 ... SP-150 (supceded by SP-2-150 (SPv6x) in 2003)
Model Type 3 ... SPx00i (generic gen2 board) (supceded by SPv6x in 2003)
Model Type 4 ... SPx004i (generic SD-2) (supceded by SD-3 in 2003)
Model Type 5 ... SP-300i (supceded by SP-2-300i (SPv6x) in 2003)
Model Type 6 ... SP-500i (supceded by SP-2-500i (SPv6x) in 2003)
Model Type 7 ... SP-750i (supceded by SP-2-750i (SPv6x) in 2003)
Model Type 8 ... INS-300 (serial number is < 42) (supceded by SPv6x based INS-300 in 2003)
Model Type 9 ... VM-502i (VM-502 with SD-2) (supceded by SD-3 based VM-502 in 2003)
Model Type 10 ... VM-504i (VM-504 with SD-2) (supceded by SD-3 based VM-504 in 2003)
Model Type 11 ... VM-503 (VM-503 with SD-2) (supceded by SD-3 based VM-503 in 2003)
Model Type 12 ... VM-505 (VM-505 with SD-2) (supceded by SD-3 based VM-505 in 2003)
Model Type 13 ... VM-506 (VM-506 with SD-2) (supceded by SD-3 based VM-506 in 2003)
Model Type 14 ... VM-507 (VM-507 with SD-2) (supceded by SD-3 based VM-507 in 2003)
Model Type 15 ... VM-510 (VM-510 with SD-2) (supceded by SD-3 based VM-510 in 2003)
Model Type 16 ... VM-511 (VM-511 with SD-2) (supceded by SD-3 based VM-511 in 2003)
Model Type 17 ... VM-512 (VM-512 with SD-2) (supceded by SD-3 based VM-512 in 2003)
Model Type 18 ... VM-521 (VM-521 with SD-2) (supceded by SD-3 based VM-521 in 2003)
Model Type 19 ... VM-523 (VM-523 with SD-2) (supceded by SD-3 based VM-523 in 2003)
Model Type 20 ... SP-275 (supceded by SP-300i in 1996)
Model Type 21 ... SP-500 (SP-500 with SD-748) (supceded by SP-500i in 1996)
Model Type 22 ... SP-750 (SP-750 with SD-748) (supceded by SP-750i in 1998)
Model Type 23 ... VM-502 (VM-502 with SD-748) (supceded by VM-502i in 1999)
Model Type 24 ... VM-504 (VM-504 with SD-748) (supceded by VM-504i in 1999)
Model Type 25 ... SD-748 (supceded by SD-2 in 1999)
Model Type 26 ... INS-150
Model Type 27 ... SPv6x (All SpectraPro's produced since 2003,
Includes: SP-2-150, SP-2-300i, SP-2-500i, SP-2-750i, SD-3,
InSight, MS-2-150i, MS-2-300i, INS-300 serial numbers > 42)
Model Type 28 ... NCL (supceded by SpectraHub in 2003)
Model Type 29 ... Filter Controller
Model Type 30 ... Reserved for Sample Chambers
Model Type 31 ... Reserved (custom, not supported by dll)
Model Type 32 ... Reserved (custom, not supported by dll)
Model Type 33 ... SpectraHub

ARC_get_Mono_preOpen_Model_int32 , returns the model type of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call.

ARC_get_ReadOut_preOpen_Model_int32, returns the model type of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call.

ARC_get_Filter_preOpen_Model_int32, returns the model type of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call.

Pre Open Serial Number Integers

ARC_get_Filter_preOpen_Serial_int32 , returns the serial number (as an integer) of a filterwheel not yet open.

ARC_get_Mono_preOpen_Serial_int32 , returns the serial number (as an integer) of a monochromator not yet open.

ARC_get_ReadOut_preOpen_Serial_int32 , returns the serial number (as an integer) of an instrument not yet open.

Serial Number Integers

ARC_get_Filter_Serial_int32 , return the serial number of a filterwheel as an integer

ARC_get_Mono_Serial_int32 , return the serial number of a monochromator as an integer

ARC_get_ReadOut_Serial_int32 , return the serial number of a readout as an integer

ARC_Search_For_Inst , function used to search for all attached Acton Research Instruments. With the exception of ARC_Ver, must be the first function called.

Delphi : function ARC_Search_For_Inst(out Num_Found : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Search_For_Inst Lib "ARC_Instrument.dll" (Num_Found As Long) As Integer

Num_Found : The number of Acton Research Instruments found and enumerated. Enumeration list starts with zero and ends with Num_Found minus one.

Result : True, if any instruments where found.

ARC_get_Num_Found_Inst_Ports , Returns the value last returned by **ARC_Search_For_Inst**. Can be called multiple times.

Delphi : function ARC_get_Num_Found_Inst_Ports : integer; stdcall;

VB : Private Declare Function ARC_get_Num_Found_Inst_Ports Lib "ARC_Instrument.dll" As Long

Result : Returns the value last returned by **ARC_Search_For_Inst**.

ARC_Open_Readout , function to open a Readout System (NCL/NCL-Lite)

Delphi : function ARC_Open_Readout(Port_Num : integer; out Enum : integer; out Mono1_Enum : integer; out Mono2_Enum : integer) : wordbool; stdcall;

VB : Private Declare Function ARC_Open_Readout Lib "ARC_Instrument.dll" (ByVal Port_Num As Long, Enum As Long, Mono1_Enum As Long, Mono2_Enum As Long) As Integer

Enum_Num : Number in the enumeration list to open.

Enum : The handle by which to address the Read Out System in all future calls. Can be validated with ARC_Valid_ReadOut_Enum.

Mono1_Enum : If a mono is attached to an NCL on port 1, then this is the enum by which to address the mono.

Mono2_Enum : If a mono is attached to an NCL on port 2, then this is the enum by which to address the second mono.

Result : True, if it successfully opened the device.

ARC_Open_Readout_Port , function to open a Readout System (NCL/NCL-Lite) on a specific COM Port (ie. COM1, COM2, ect...)

Delphi : function ARC_Open_Readout_Port(Com_Num : integer; out Enum : integer; out Mono1_Enum : integer; out Mono2_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Open_Readout_Port Lib "ARC_Instrument.dll" (ByVal Com_Num As Long, Enum As Long, Mono1_Enum As Long, Mono2_Enum As Long) As Integer

Com_Num : Com Port (COM1,COM2,ect.) to open.

Enum : The handle by which to address the Read Out System in all future calls. Can be validated with ARC_Valid_ReadOut_Enum.

Mono1_Enum : If a mono is attached to an NCL on port 1, then this is the enum by which to address the mono.

Mono2_Enum : If a mono is attached to an NCL on port 2, then this is the enum by which to address the second mono.

Result : True, if it successfully opened the device.

ARC_Close_Enum , function to close an open enumeration.

Delphi : function ARC_Close_Enum(Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Close_Enum Lib "ARC_Instrument.dll" (ByVal Enum As Long) As Integer

Enum : The handle defined in ARC_Open_xxxx by which the instrument is to be addressed.

Result : True, if the operations was carried out successfully.

ARC_Valid_Enum , function returns is an enumeration is both valid and the instrument is open.

Delphi : function ARC_Valid_Enum(Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Valid_Enum Lib "ARC_Instrument.dll" (ByVal Enum As Long) As Integer

Enum : The handle defined in ARC_Open_xxxx by which the instrument is to be addressed.

Result : True if Readout_Enum is a valid Monochromator that is open.

ARC_Valid_Readout_Enum , function returns is a Readout System (NCL/NCL-Lite) enumeration is both valid and the instrument is open.

Delphi : function ARC_Valid_Readout_Enum(Readout_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Valid_Readout_Enum Lib "ARC_Instrument.dll" (ByVal Readout_Enum As Long) As Integer

Readout_Enum : The handle defined in ARC_Open_Readout by which the readout is to be addressed.

Result : True if Readout_Enum is a valid Monochromator that is open.

ARC_get_Readout_preOpen_Model , returns the model string of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call. In the case of multiple Readout Systems attached, it allows the user to sort, which instruments are to be opened before opening them.

Delphi : function ARC_get_Readout_preOpen_Model(Num : integer; out Model_Var : olevariant) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Readout_preOpen_Model Lib "ARC_Instrument.dll" (ByVal Enum As Long, Model_Var As Variant) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Model_Var : The model string of the unopened monochromator.

Result : True if the operation was successful

ARC_Send_CMD_To_Inst , this is an internal function that provides direct communications with the instrument. Usage of this function should be avoided.

Delphi : function ARC_Send_CMD_To_Inst(Enum : integer; SendCMD : olevariant; out RCV_Var : olevariant; ms_Timeout : integer) : wordbool; stdcall;

VB : Private Declare Function ARC_Send_CMD_To_Inst Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal SendCMD As Variant, RCV_Var As Variant, ByVal ms_Timeout As Long) As Integer

Enum : The handle defined in ARC_Open_xxxx by which the instrument is to be addressed.

SendCMD : The text command to be sent to the instrument. Note, line termination is handled by the DLL.

RCV_Var : The reply from the instrument to the given command.

Ms_Timeout : How long the DLL should wait in milliseconds for an acknowledgement before returning

Result : True if the instrument did not reject the command or timeout.

ARC_Ver , this function provides the DLL version. It is the only function that can be called at any time.

Delphi : function ARC_Ver(out Major : integer; out Minor : integer; out Build : integer) : wordbool; stdcall;

VB : Private Declare Function ARC_Ver Lib "ARC_Instrument.dll" (Majorval As Long, Minorval As Long, Buildval As Long) As Integer

Major : Version Major Number

Minor : Version Minor Number

Build : Version Build Number

Result : True if the operation was successful

ARC_get_ReadOut_Model , returns the model string from the instrument

Delphi : function ARC_get_Readout_Model(Enum : integer;out Model_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_ReadOut_Model Lib "ARC_Instrument.dll" (ByVal Enum As Long,
Model_str As Variant) As Integer

Enum : The handle defined in ARC_Open_Readout by which the instrument is to be addressed.

Model_Var : Returns the model string of the instrument.

Result : True if the operation was successful

ARC_get_ReadOut_Serial , returns the serial number from the instrument

Delphi : function ARC_get_ReadOut_Serial(Enum : integer;out Serial_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_ReadOut_Serial Lib "ARC_Instrument.dll" (ByVal Enum As Long, Serial_str As Variant) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Serial_Var : Returns the serial number of the instrument as a string.

Result : True if the operation was successful

ARC_get_Num_Det, return the number of single point detectors in the Readout System

Delphi : function ARC_get_Num_Det(Enum : integer): integer; stdcall;

VB : Private Declare Function ARC_get_Num_Det Lib "ARC_Instrument.dll" (ByVal Enum As Long) As Long

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : Number of single point detectors the readout system supports. (1 NCL-Lite, 2,3 NCL)

ARC_Valid_Det_Num, return if the detector number is valid on the Readout System

Delphi : function ARC_Valid_Det_Num(Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Valid_Det_Num Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector number we are checking

Result : True if the operation was successful

ARC_get_ReadOut_ITime_ms, return the integration time used for taking a reading

Delphi : function ARC_get_ReadOut_ITime_ms(Enum : integer): integer; stdcall;

VB : Private Declare Function ARC_get_ReadOut_ITime_ms Lib "ARC_Instrument.dll" (ByVal Enum As Long) As Long

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : The integration time used for taking a reading in milliseconds.

ARC_set_ReadOut_ITime_ms, set the integration time used for taking a reading

Delphi : function ARC_set_ReadOut_ITime_ms(Enum : integer; ITime_ms : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_ReadOut_ITime_ms Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal ITime_ms As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

ITime_ms : The integration time in milliseconds to be used when reading out a detector

Result : True if the operation was successful

ARC_get_Det_Type, return the detector readout mode (Current,Voltage,Photon Counting)

Delphi : function ARC_get_Det_Type(Enum : integer; Det_Num : integer; out DetType : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Det_Type Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, DetType As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

DetType : The detector readout type

Detector Type 1 ... Current

Detector Type 2 ... Voltage

Detector Type 3 ... Photon Counting

Result : True if the operation was successful

ARC_get_Det_Type_Str, return a variant string of the detector readout mode

Delphi : function ARC_get_Det_Type_Str(Enum : integer; Det_Num : integer; out DetType : olevariant) : wordbool;
stdcall;

VB : Private Declare Function ARC_get_Det_Type_Str Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal
Det_Num As Long, DetType As Variant) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

DetType : The detector readout type (Current,Voltage,Photon Counting)

Result : True if the operation was successful

ARC_set_Det_Type, set the detector readout type

Delphi : function ARC_set_Det_Type(Enum : integer; Det_Num : integer; DetType : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_set_Det_Type Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, ByVal DetType As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

DetType : The detector readout type

Detector Type 1 ... Current

Detector Type 2 ... Voltage

Detector Type 3 ... Photon Counting

Result : True if the operation was successful

ARC_set_Det_Photon, set the detector readout type to photon counter

Delphi : function ARC_set_Det_Photon (Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_set_Det_Photon Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the operation was successful

ARC_set_Det_Voltage, set the detector readout type to voltage

Delphi : function ARC_set_Det_Voltage (Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_set_Det_VoltageLib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the operation was successful

ARC_set_Det_Current, set the detector readout type to current

Delphi : function ARC_set_Det_Current (Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_set_Det_Current Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the operation was successful

ARC_get_Det_BiPolar, return if a detector takes bipolar (+/-) readings

Delphi : function ARC_get_Det_BiPolar(Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_get_Det_BiPolar Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the detector is BiPolar (+/-)

ARC_get_Det_BiPolar_Str, return a variant string of the detector polarity

Delphi : function ARC_get_Det_BiPolar_Str(Enum : integer; Det_Num : integer; out BiPolarStr : olevariant):

wordbool; stdcall;

VB : Private Declare ARC_get_Det_BiPolar_Str Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, BiPolarStr As Variant) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

BiPolarStr : returns if the detector is uni-polar or bipolar as a variant string

Result : True if the operation was successful

ARC_set_Det_BiPolar, set the detector readout to bipolar (+/-)

Delphi : function ARC_set_Det_BiPolar(Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_set_Det_BiPolar Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the operation was successful

ARC_set_Det_UniPolar, set the detector readout to unipolar (-)

Delphi : function ARC_set_Det_UniPolar (Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_set_Det_UniPolar Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the operation was successful

ARC_get_Det_Range, return the detector range factor

Delphi : function ARC_get_Det_Range(Enum : integer; Det_Num : integer; out Range : integer) : wordbool; stdcall;
VB : Private Declare ARC_get_Det_Range Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, Range As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Range : The detector gain range

Detector Range 0 ... 1x

Detector Range 1 ... 2x

Detector Range 2 ... 4x

Detector Range 3 ... 50x

Detector Range 4 ... 200x

Result : True if the operation was successful

ARC_get_Det_Range_Factor, return the detector range multiplier

Delphi : function ARC_get_Det_Range_Factor(Enum : integer; Det_Num : integer; out Range_Factor : integer) : wordbool; stdcall;

VB : Private Declare ARC_get_Det_Range Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, Range_Factor As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Range_Factor : The detector gain

Result : True if the operation was successful

ARC_set_Det_Range, set the detector range factor

Delphi : function ARC_set_Det_Range(Enum : integer; Det_Num : integer; Range : integer) : wordbool; stdcall;
VB : Private Declare ARC_set_Det_Range Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, ByVal Range As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Range : The detector gain range

Detector Range 0 ... 1x

Detector Range 1 ... 2x

Detector Range 2 ... 4x

Detector Range 3 ... 50x

Detector Range 4 ... 200x

Result : True if the operation was successful

ARC_get_Det_HVon, return if the high voltage for a detector is turned on

Delphi : function ARC_get_Det_HVon(Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_get_Det_HVon Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the high voltage is turned on

ARC_set_Det_HV_on, set the detector high voltage to on

Delphi : function ARC_set_Det_HV_on(Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_set_Det_HV_on Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the high voltage has been turned on

ARC_set_Det_HV_off, set the detector high voltage to off

Delphi : function ARC_set_Det_HV_off(Enum : integer; Det_Num : integer): wordbool; stdcall;

VB : Private Declare ARC_set_Det_HV_off Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the high voltage has been turned off

ARC_get_Det_HV_Volts, return the high voltage volts setting

Delphi : function ARC_get_Det_HV_Volts(Enum : integer; Det_Num : integer): integer; stdcall;

VB : Private Declare ARC_get_Det_HV_Volts Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long) As Long

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : High Voltage Volts that the detector is set to.

ARC_set_Det_HV_Volts, set the voltage value of the high voltage

Delphi : function ARC_set_Det_HV_Volts(Enum : integer; Det_Num : integer; HV_Volts : integer): wordbool; stdcall;
VB : Private Declare ARC_set_Det_HV_Volts Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, ByVal HV_Volts As Long) As Long

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

HV_Volts :

Result : True if the operation was successful

ARC_get_Det_NumAvgRead, return the number of readings that are averaged

Delphi : function ARC_get_Det_NumAvgRead(Enum : integer) : integer; stdcall;

VB : Private Declare ARC_get_Det_NumAvgRead Lib "ARC_Instrument.dll" (ByVal Enum As Long) As Long

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : Number of readings averaged in into a single reading

ARC_set_Det_NumAvgRead, set the number of readings to average

Delphi : function ARC_set_Det_NumAvgRead(Enum : integer; NumReadsAvg : integer): wordbool; stdcall;
VB : Private Declare ARC_get_Det_NumAvgRead Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal NumReadsAvg As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

NumReadsAvg : The number reads that will be required to acquire a single read.

Result : True if the operation was successful

ARC_Det_Read, readout a single detector

Delphi : function ARC_Det_Read(Enum : integer; Det_Num : integer; out Read_Value : double) : wordbool; stdcall;
VB : Private Declare ARC_Det_Read Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, Read_Value As Double) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Read_Value : Reading Value for the selected detector

Result : True if the operation was successful

ARC_Det_ReadAll, readout all detectors

Delphi : function ARC_Det_ReadAll(Enum : integer; out Det1_Value, Det2_Value, Det3_Value : double) : wordbool; stdcall;

VB : Private Declare ARC_Det_Read Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Det_Num As Long, Det1_Value As Double, Det2_Value As Double, Det3_Value As Double) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det1_Value : The value of detector 1

Det2_Value : The value of detector 2

Det3_Value : The value of detector 3

Result : True if the operation was successful

ARC_Det_Start_NonBlock_Read , start to readout a single detector, but return immediately

Delphi : function ARC_Det_Start_NonBlock_Read(Enum : integer; Det_Num : integer) : wordbool; stdcall;

VB : Private Declare ARC_Det_Start_NonBlock_Read(ByVal Enum As Long , ByVal Det_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Result : True if the operation was successful

ARC_Det_NonBlock_Read_Done , determine is a single detector readout has completed and return value

Delphi : function ARC_Det_NonBlock_Read_Done (Enum : integer; Det_Num : integer; out Read_Value : double) : wordbool; stdcall;
VB : Private Declare ARC_Det_NonBlock_Read_Done(ByVal Enum As Long , ByVal Det_Num As Long , Read_Value As Double) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Det_Num : The detector to be addressed

Read_Value : The Value of the last Non-Blocking Read. Is zero if the read has not completed.

Result : True if the read has completed. False if still reading.

ARC_get_NCL_Filter_Present, returns if the instrument has an integrated filter wheel setup

Delphi : function ARC_get_NCL_Filter_Present(Enum : integer): wordbool; stdcall;

VB : Private Declare ARC_get_NCL_Filter_Present Lib "ARC_Instrument.dll" (ByVal Enum As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : If the NCL Filter Wheel is setup and present.

ARC_set_NCL_Filter_Present, set the instrument filter wheel to active (NCL only)

Delphi : function ARC_set_NCL_Filter_Present(Enum : integer; Min_Filter : integer; Max_Filter : integer): wordbool; stdcall;

VB : Private Declare ARC_set_NCL_Filter_Present Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Min_Filter As Long, ByVal Max_Filter As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Min_Filter : Min Filter Position (on an NCL with the standard filter wheel 1)

Max_Filter : Max Filter Position (on an NCL with the standard filter wheel 6)

Result : True if the operation was successful

ARC_get_NCL_Filter_Position, return the current filter position

Delphi : function ARC_get_NCL_Filter_Position(Enum: integer; out Position : integer): wordbool; stdcall;

VB : Private Declare ARC_get_NCL_Filter_Position Lib "ARC_Instrument.dll" (ByVal Enum As Long, Position As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Position : The current filter position

Result : True if the operation was successful

ARC_set_NCL_Filter_Position, set the current filter position

Delphi : function ARC_set_NCL_Filter_Position(Enum: integer; Position : integer): wordbool; stdcall;

VB : Private Declare ARC_set_NCL_Filter_Position Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Position As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Position : The current filter position

Result : True if the operation was successful

ARC_get_NCL_Filter_Min_Pos, returns the min. filter position

Delphi : function ARC_get_NCL_Filter_Min_Pos(Enum : integer; out Min_Pos : integer): wordbool; stdcall;
VB : Private Declare ARC_set_NCL_Filter_Min_Pos Lib "ARC_Instrument.dll" (ByVal Enum As Long, Min_Pos As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Min_Pos : The min filter position

Result : True if the operation was successful

ARC_get_NCL_Filter_Max_Pos, returns the max. filter position

Delphi : function ARC_get_NCL_Filter_Max_Pos(Enum : integer; out Max_Pos : integer): wordbool; stdcall;

VB : Private Declare ARC_set_NCL_Filter_Max_Pos Lib "ARC_Instrument.dll" (ByVal Enum As Long, Max_Pos As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Max_Pos : The max filter position

Result : True if the operation was successful

ARC_NCL_Filter_Home, home the filter wheel

Delphi : function ARC_NCL_Filter_Home(NCL_Enum : integer): wordbool; stdcall;

VB : Private Declare ARC_set_NCL_Filter_Home Lib "ARC_Instrument.dll" (ByVal Enum As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True if the operation was successful

ARC_get_NCL_Shutter_Valid, return if a NCL Readout Shutter number is valid.

Delphi : function ARC_get_NCL_Shutter_Valid(Enum : integer; Shutter_Num : integer) : wordbool; stdcall;
VB : Private Declare ARC_get_NCL_Shutter_Valid Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Shutter_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Shutter_Num : The shutter being addressed (1 or 2 on an NCL)

Result : True if the shutter is valid

ARC_get_NCL_Shutter_Open, return if a NCL Readout shutter is open

Delphi : function ARC_get_NCL_Shutter_Open(Enum : integer; Shutter_Num : integer) : wordbool; stdcall;
VB : Private Declare ARC_get_NCL_Shutter_Open Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Shutter_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Shutter_Num : The shutter being addressed (1 or 2 on an NCL)

Result : True if the shutter is in the open position

ARC_set_NCL_Shutter_Open, set a NCL Readout shutter to a closed state

Delphi : function ARC_set_NCL_Shutter_Open(Enum : integer; Shutter_Num : integer) : wordbool; stdcall;
VB : Private Declare ARC_set_NCL_Shutter_Open Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Shutter_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Shutter_Num : The shutter being addressed (1 or 2 on an NCL)

Result : True if the operation was successful

ARC_set_NCL_Shutter_Closed, set a NCL Readout shutter to a closed state

Delphi : function ARC_set_NCL_Shutter_Closed(Enum : integer; Shutter_Num : integer) : wordbool; stdcall;
VB : Private Declare ARC_set_NCL_Shutter_Closed Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Shutter_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Shutter_Num : The shutter being addressed (1 or 2 on an NCL)

Result : True if the operation was successful

ARC_get_SpectraHub_Shutter_Enabled, return if a SpectraHub Shutter number is enabled

Delphi : function ARC_get_SpectraHub_Shutter_Enabled(Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_get_SpectraHub_Shutter_Enabled Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter has been enabled by ARC_set_SpectraHub_Shutter_Enable.

ARC_set_SpectraHub_Shutter_Enable , enable the shutter port on a SpectraHub

Delphi : function ARC_set_SpectraHub_Shutter_Enable (Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_SpectraHub_Shutter_Enable Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter has been enabled.

ARC_set_SpectraHub_Shutter_Disable, disable the shutter port on a SpectraHub

Delphi : function ARC_set_SpectraHub_Shutter_Disable (Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_SpectraHub_Shutter_Disable Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter has been disabled.

ARC_get_SpectraHub_Shutter_Open_High, return if the shutter port on a SpectaHub opens on TTL high

Delphi : function ARC_get_SpectraHub_Shutter_Open_High (Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_get_SpectraHub_Shutter_Open_High Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter opens on a TTL high.

ARC_set_SpectraHub_Shutter_Open_High, sets the SpectraHub shutter port to use TTL high to open

Delphi : function ARC_set_SpectraHub_Shutter_Open_High (Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_SpectraHub_Shutter_Open_High Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter has been set to open on a TTL high.

ARC_set_SpectraHub_Shutter_Open_Low , sets the SpectraHub shutter port to use TTL low to open

Delphi : function ARC_set_SpectraHub_Shutter_Open_Low (Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_SpectraHub_Shutter_Open_Low Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter has been set to open on a TTL low.

ARC_get_SpectraHub_Shutter_Open, return of the SpectraHub shutter is open

Delphi : function ARC_get_SpectraHub_Shutter_Open (Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_get_SpectraHub_Shutter_Open Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter is open.

ARC_set_SpectraHub_Shutter_Open, Open the SpectraHub shutter

Delphi : function ARC_set_SpectraHub_Shutter_Open (Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_SpectraHub_Shutter_Open Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter has been opened.

ARC_set_SpectraHub_Shutter_Closed, Close the SpectraHub shutter

Delphi : function ARC_set_SpectraHub_Shutter_Closed (Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_SpectraHub_Shutter_Closed Lib "ARC_Instrument.dll" (ByVal Hub_Enum as Long) as integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the shutter has been closed.

ARC_get_NCL_TTL_Valid, return if a TTL line is a valid line

Delphi : function ARC_get_NCL_TTL_Valid(Enum : integer; TTL_Line : integer) : wordbool; stdcall;

VB : Private Declare ARC_get_NCL_TTL_Valid Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal TTL_Line As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

TTL_Line : The TTL line number being addressed

Result : True if the TTL_Line is a valid line number

ARC_get_NCL_TTL_In, return if an input TTL line is being pulled on by an outside connection

Delphi : function ARC_get_NCL_TTL_In(Enum : integer; TTL_Line : integer) : wordbool; stdcall;

VB : Private Declare ARC_get_NCL_TTL_In Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal TTL_Line As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

TTL_Line : The TTL line number being addressed

Result : True if the input TTL_Line is being pulled to TTL high

ARC_get_NCL_TTL_Out, return if an output TTL line is on

Delphi : function ARC_get_NCL_TTL_Out(Enum : integer; TTL_Line : integer) : wordbool; stdcall;

VB : Private Declare ARC_get_NCL_TTL_Out Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal TTL_Line As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

TTL_Line : The TTL line number being addressed

Result : True if the output TTL_Line is set to TTL high

ARC_set_NCL_TTL_Out_On, turn a TTL line on

Delphi : function ARC_set_NCL_TTL_Out_On(Enum : integer; TTL_Line : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_NCL_TTL_Out_On Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal TTL_Line As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

TTL_Line : The TTL line number being addressed

Result : True if the operation was successful

ARC_set_NCL_TTL_Out_Off, turn a TTL line off

Delphi : function ARC_set_NCL_TTL_Out_Off(Enum : integer; TTL_Line : integer) : wordbool; stdcall;
VB : Private Declare ARC_set_NCL_TTL_Out_Off Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal
TTL_Line As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

TTL_Line : The TTL line number being addressed

Result : True if the operation was successful

ARC_get_SpectraHub_Trig_On, return if reading on a SpectraHub wait for a trigger before starting

Delphi : function ARC_get_SpectraHub_Trig_On(Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_get_SpectraHub_Trig_On Lib "ARC_Instrument.dll" (ByVal Hub_Enum As Long) As integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the read trigger is active.

ARC_set_SpectraHub_Trig_On, set SpectraHub readings to wait for a trigger before starting

Delphi : function ARC_set_SpectraHub_Trig_On(Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_SpectraHub_Trig_On Lib "ARC_Instrument.dll" (ByVal Hub_Enum As Long) As integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the read trigger is set active.

ARC_set_SpectraHub_Trig_Off, turn off SpectraHub readings waiting for a trigger before starting

Delphi : function ARC_set_SpectraHub_Trig_Off(Hub_Enum : integer) : wordbool; stdcall;

VB : Private Declare ARC_set_SpectraHub_Trig_Off Lib "ARC_Instrument.dll" (ByVal Hub_Enum As Long) As integer

Hub_Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Result : True (<> 0) if the read trigger is turned off.

ARC_get_NCL_Mono_Setup, return if the open NCL port has a Monochromator attached

Delphi : function ARC_get_NCL_Mono_Setup(Enum : integer; Mono_Num : integer): wordbool; stdcall;
VB : Private Declare ARC_get_NCL_Mono_Setup Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Mono_Num As Long) As Integer

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Mono_Num : The readout system monochromator port being addressed

Result : True if a mono is attached to the port and setup

ARC_get_NCL_Mono_Enum, return the enumeration of the attached Monochromator

Delphi : function ARC_get_NCL_Mono_Enum(Enum : integer; Mono_Num : integer): integer; stdcall;

VB : Private Declare ARC_get_NCL_Mono_Enum Lib "ARC_Instrument.dll" (ByVal Enum As Long, ByVal Mono_Num As Long) As Long

Enum : The handle defined in ARC_Open_Readout by which the readout system is to be addressed.

Mono_Num : The readout system monochromator port being addressed

Result : The enumeration of the monochromator. Check the value returned with ARC_Valid_Mono_Enum

ARC_Open_Mono , function to open a monochromator

Delphi : function ARC_Open_Mono(Enum_Num : integer; out Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Open_Mono Lib "ARC_Instrument.dll" (ByVal Enum_Num As Long,
Mono_Enum As Long) As Integer

Enum_Num : Number in the enumeration list to open.

Mono_Enum : The handle by which to address the monochromator in all future calls. Can be validated with
ARC_Valid_Mono_Enum.

Result : True, if it successfully opened the device.

ARC_Open_Mono_Port , function to open a monochromator on a specific COM Port (ie. COM1, COM2, ect...)

Delphi : function ARC_Open_Mono_Port(Com_Num : integer; out Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Open_Mono_Port Lib "ARC_Instrument.dll" (ByVal Com_Num As Long, Mono_Enum As Long) As Integer

Com_Num : Com Port (COM1,COM2,ect.) to open.

Mono_Enum : The handle by which to address the monochromator in all future calls. Can be validated with ARC_Valid_Mono_Enum.

Result : True, if it successfully opened the device.

ARC_Valid_Mono_Enum , function returns if a monochromator enumeration is both valid and the instrument is open. All functions call this function to verify that the requested action is valid.

Delphi : function ARC_Valid_Mono_Enum(Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Valid_Mono_Enum Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if Mono_Enum is a valid Monochromator that is open.

ARC_get_Mono_preOpen_Model , returns the model string of an instrument not yet opened. Note, ARC_Search_For_Mono needs to be called prior to this call. In the case of multiple Monochromator/Spectrographs attached, it allows the user to sort, which instruments are to be opened before opening them.

Delphi : function ARC_get_Mono_preOpen_Model(Enum_Num : integer; out Model_Var : olevariant) : wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_preOpen_Model Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Model_Var As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Model_Var : The model string of the unopened monochromator.

Result : True if the operation was successful

ARC_get_Mono_Model , returns the model string from the instrument

Delphi : function ARC_get_Mono_Model(Mono_Enum : integer;out Model_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Model Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long,
Model_str As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Model_Var : Returns the model string of the instrument.

Result : True if the operation was successful

ARC_get_Mono_Serial , returns the serial number from the instrument

Delphi : function ARC_get_Mono_Serial(Mono_Enum : integer;out Serial_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Serial Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long,
Serial_str As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Serial_Var : Returns the serial number of the instrument as a string.

Result : True if the operation was successful

ARC_get_Mono_Focallength , returns the default focal length of the instrument

Delphi : function ARC_get_Mono_Focallength(Mono_Enum : integer; out Focallength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Focallength Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Focallength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Focallength : Returns the focal length of the instrument in millimeters.

Result : True if the operation was successful

ARC_get_Mono_HalfAngle , returns the default half angle of the instrument in radians

Delphi : function ARC_get_Mono_HalfAngle(Mono_Enum : integer; out HalfAngle : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_HalfAngle Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, HalfAngle As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

HalfAngle : Returns the halfangle of the instrument in radians.

Result : True if the operation was successful

ARC_get_Mono_DetectorAngle , returns the default detector angle of the instrument in radians

Delphi : function ARC_get_Mono_DetectorAngle(Mono_Enum : integer; out DetectorAngle : double): wordbool;
stdcall;

VB : Private Declare Function ARC_get_Mono_DetectorAngle Lib "ARC_Instrument.dll" (ByVal Mono_Enum As
Long, DetectorAngle As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

DetectorAngle : Returns the default detector angle of the instrument in radians.

Result : True if the operation was successful

ARC_get_Mono_Double , returns if the instrument is a double monochromator

Delphi : function ARC_get_Mono_Double(Mono_Enum : integer; out Double_Present : wordbool): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Double Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long,
Double_Present As Integer) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Double_Present : Returns true if the instrument is a double monochromator.

Result : True if the operation was successful

ARC_get_Mono_Double_Subtractive , returns if a double monochromator is subtractive instead of additive

Delphi : function ARC_get_Mono_Double_Subtractive(Mono_Enum : integer;out Double_Subtractive : wordbool): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Double_Subtractive Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Double_Subtractive As Integer) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Double_Present : Returns true (<> 0) if the instrument is a double subtractive monochromator.

Result : True (<> 0) if the operation was successful

ARC_get_Mono_Precision , returns the nm decimal precision of the wavelength drive. Note this is independent of the wavelength step resolution whose coarseness is defined by the grating being used.

Delphi : function ARC_get_Mono_Precision(Mono_Enum : integer): integer; stdcall;

VB : Private Declare Function ARC_get_Mono_Precision Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Long

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : The number of digits after the decimal point the instrument uses. Note, the true precision is limited by the density of the grating and can be much less than the instruments wavelength precision.

ARC_get_Mono_Backlash_Steps, returns the number of backlash steps used when reversing the wavelength drive

Delphi : function ARC_get_Mono_Backlash_Steps(Mono_Enum : integer; out Backlash : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Backlash_Steps Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Backlash As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Backlash : Returns the number of steps the instrument backlash corrects. Not valid on older instruments.

Result : True if the operation was successful

ARC_get_Mono_Gear_Steps, returns the number of steps in a set of sine drive gears

Delphi : function ARC_get_Mono_Gear_Steps(Mono_Enum : integer; out minor_steps, major_steps : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Gear_Steps Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, minor_steps As Long, major_steps As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Minor_steps : Returns the number of steps per rev on the minor gear of a sine wavelength drive.

Major_steps : Returns the number of steps per rev on the major gear of a sine wavelength drive.

Result : True if the operation was successful

ARC_get_Mono_nmRev_Ratio, returns the number of stepper steps per rev of the wavelength drive motor

Delphi : function ARC_get_Mono_nmRev_Ratio (Mono_Enum : integer; out nmRev_Ratio : double): wordbool;
stdcall;

VB : Private Declare Function ARC_get_Mono_nmRev_Ratio Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, nmRev_Ratio As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

nmRev_Ratio : Returns the ratio of nm per rev in a linear wavelength drive.

Result : True if the operation was successful

ARC_get_Mono_Pixel_Map_nm , returns the current pixel wavelength map in nm

Delphi : function ARC_get_Mono_Pixel_Map_nm (Mono_Enum : integer; Pixel_Size_um : double;
Number_X_Pixels : integer; Left_Adj_Percent : double; Center_Adj_Pixels : double; Right_Adj_Percent : double;
Pixel_Map : Pointer) : wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Pixel_Map_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Pixel_Size_um As Double, ByVal Number_X_Pixels As Long, ByVal Left_Adj_Percent As Double, ByVal Center_Adj_Pixels As Double, ByVal Right_Adj_Percent As Double, Pixel_Map As Any) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Pixel_Size_um : CCD Pixel width in microns

Number_X_Pixels : Number of CCD Pixels in the X direction

Left_Adj_Percent : Percent adjust to the left edge (default 0.0)

Center_Adj_Pixels : Number of pixels from center the CCD is (default 0.0)

Right_Adj_Percent : Percent adjust to the right edge (default 0.0)

Pixel_Map : The pixel to wavelength mapping array. This is an array of double that is larger than the number of pixels in the X direction.

Result : True (<> 0) if the operation was successful

ARC_get_Mono_Pixel_Map_ang , returns the current pixel wavelength map in Angstroms

Delphi : function ARC_get_Mono_Pixel_Map_ang (Mono_Enum : integer; Pixel_Size_um : double;
Number_X_Pixels : integer; Left_Adj_Percent : double; Center_Adj_Pixels : double; Right_Adj_Percent : double;
Pixel_Map : Pointer) : wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Pixel_Map_ang Lib "ARC_Instrument.dll" (ByVal Mono_Enum As
Long, ByVal Pixel_Size_um As Double, ByVal Number_X_Pixels As Long, ByVal Left_Adj_Percent As Double,
ByVal Center_Adj_Pixels As Double, ByVal Right_Adj_Percent As Double, Pixel_Map As Any) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Pixel_Size_um : CCD Pixel width in microns

Number_X_Pixels : Number of CCD Pixels in the X direction

Left_Adj_Percent : Percent adjust to the left edge (default 0.0)

Center_Adj_Pixels : Number of pixels from center the CCD is (default 0.0)

Right_Adj_Percent : Percent adjust to the right edge (default 0.0)

Pixel_Map : The pixel to wavelength mapping array. This is an array of double that is larger than the number of
pixels in the X direction.

Result : True (<> 0) if the operation was successful

ARC_get_Mono_Pixel_Map_eV , returns the current pixel wavelength map in electron Volts

Delphi : function ARC_get_Mono_Pixel_Map_eV (Mono_Enum : integer; Pixel_Size_um : double;
Number_X_Pixels : integer; Left_Adj_Percent : double; Center_Adj_Pixels : double; Right_Adj_Percent : double;
Pixel_Map : Pointer) : wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Pixel_Map_eV Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Pixel_Size_um As Double, ByVal Number_X_Pixels As Long, ByVal Left_Adj_Percent As Double, ByVal Center_Adj_Pixels As Double, ByVal Right_Adj_Percent As Double, Pixel_Map As Any) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Pixel_Size_um : CCD Pixel width in microns

Number_X_Pixels : Number of CCD Pixels in the X direction

Left_Adj_Percent : Percent adjust to the left edge (default 0.0)

Center_Adj_Pixels : Number of pixels from center the CCD is (default 0.0)

Right_Adj_Percent : Percent adjust to the right edge (default 0.0)

Pixel_Map : The pixel to wavelength mapping array. This is an array of double that is larger than the number of pixels in the X direction.

Result : True (<> 0) if the operation was successful

ARC_get_Mono_Pixel_Map_micron , returns the current pixel wavelength map in microns

Delphi : function ARC_get_Mono_Pixel_Map_micron (Mono_Enum : integer; Pixel_Size_um : double;
Number_X_Pixels : integer; Left_Adj_Percent : double; Center_Adj_Pixels : double; Right_Adj_Percent : double;
Pixel_Map : Pointer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Pixel_Map_micron Lib "ARC_Instrument.dll" (ByVal Mono_Enum
As Long, ByVal Pixel_Size_um As Double, ByVal Number_X_Pixels As Long, ByVal Left_Adj_Percent As Double,
ByVal Center_Adj_Pixels As Double, ByVal Right_Adj_Percent As Double, Pixel_Map As Any) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Pixel_Size_um : CCD Pixel width in microns

Number_X_Pixels : Number of CCD Pixels in the X direction

Left_Adj_Percent : Percent adjust to the left edge (default 0.0)

Center_Adj_Pixels : Number of pixels from center the CCD is (default 0.0)

Right_Adj_Percent : Percent adjust to the right edge (default 0.0)

Pixel_Map : The pixel to wavelength mapping array. This is an array of double that is larger than the number of pixels in the X direction.

Result : True (<> 0) if the operation was successful

ARC_get_Mono_Pixel_Map_absCM , returns the current pixel wavelength map in absolute wavenumber

Delphi : function ARC_get_Mono_Pixel_Map_absCM (Mono_Enum : integer; Pixel_Size_um : double; Number_X_Pixels : integer; Left_Adj_Percent : double; Center_Adj_Pixels : double; Right_Adj_Percent : double; Pixel_Map : Pointer) : wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Pixel_Map_absCM Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Pixel_Size_um As Double, ByVal Number_X_Pixels As Long, ByVal Left_Adj_Percent As Double, ByVal Center_Adj_Pixels As Double, ByVal Right_Adj_Percent As Double, Pixel_Map As Any) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Pixel_Size_um : CCD Pixel width in microns

Number_X_Pixels : Number of CCD Pixels in the X direction

Left_Adj_Percent : Percent adjust to the left edge (default 0.0)

Center_Adj_Pixels : Number of pixels from center the CCD is (default 0.0)

Right_Adj_Percent : Percent adjust to the right edge (default 0.0)

Pixel_Map : The pixel to wavelength mapping array. This is an array of double that is larger than the number of pixels in the X direction.

Result : True (<> 0) if the operation was successful

ARC_get_Mono_Pixel_Map_relCM , returns the current pixel wavelength map in relative wavenumber

Delphi : function ARC_get_Mono_Pixel_Map_nm (Mono_Enum : integer; Pixel_Size_um : double;
Number_X_Pixels : integer; Laser_Line_nm : double; Left_Adj_Percent : double; Center_Adj_Pixels : double;
Right_Adj_Percent : double; Pixel_Map : Pointer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Pixel_Map_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As
Long, ByVal Pixel_Size_um As Double, ByVal Number_X_Pixels As Long, ByVal Laser_Line_nm As Double, ByVal
Left_Adj_Percent As Double, ByVal Center_Adj_Pixels As Double, ByVal Right_Adj_Percent As Double,
Pixel_Map As Any) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Pixel_Size_um : CCD Pixel width in microns

Number_X_Pixels : Number of CCD Pixels in the X direction

Laser_Line_nm : The laser line in nm that the wavenumber is in relation to. Note never use 0.0, that implies a
laser of infinite photon energy. No lasers currently exist below 100 nm. 100.0 should be a lower limit on this value.

Left_Adj_Percent : Percent adjust to the left edge (default 0.0)

Center_Adj_Pixels : Number of pixels from center the CCD is (default 0.0)

Right_Adj_Percent : Percent adjust to the right edge (default 0.0)

Pixel_Map : The pixel to wavelength mapping array. This is an array of double that is larger than the number of
pixels in the X direction.

Result : True (<> 0) if the operation was successful

ARC_get_Mono_Wavelength_nm , returns the current center wavelength of instrument in nm.

Delphi : function ARC_get_Mono_Wavelength_nm(Mono_Enum : integer; out Wavelength : double): wordbool;
stdcall;

VB : Private Declare Function ARC_get_Mono_Wavelength_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The current wavelength of the instrument in nanometers.

Result : True if the operation was successful

ARC_set_Mono_Wavelength_nm , sets the center wavelength of the instrument in nm.

Delphi : function ARC_set_Mono_Wavelength_nm(Mono_Enum : integer; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The wavelength in nm that the instrument is to be moved to.

Result : True if the operation was successful

ARC_get_Mono_Wavelength_ang , returns the current center wavelength of instrument in Angstroms. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_ang(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Wavelength_ang Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The current wavelength of the instrument in Angstroms.

Result : True if the operation was successful

ARC_set_Mono_Wavelength_ang , sets the center wavelength of the instrument in angstroms. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_ang(Mono_Enum : integer; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_ang Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The wavelength in Angstroms that the instrument is to be moved to.

Result : True if the operation was successful

ARC_get_Mono_Wavelength_eV , returns the current center wavelength of instrument in electron Volts.
Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_eV(Mono_Enum : integer; out Wavelength : double): wordbool;
stdcall;

VB : Private Declare Function ARC_get_Mono_Wavelength_eV Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The current wavelength of the instrument in Electron Volts.

Result : True if the operation was successful

ARC_set_Mono_Wavelength_eV , sets the center wavelength of the instrument in electron Volts. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_eV(Mono_Enum : integer; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_eV Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The wavelength in Electron Volts that the instrument is to be moved to.

Result : True if the operation was successful

ARC_get_Mono_Wavelength_micron , returns the current center wavelength of instrument in microns. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_micron(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Wavelength_micron Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The current wavelength of the instrument in microns.

Result : True if the operation was successful

ARC_set_Mono_Wavelength_micron , sets the center wavelength of the instrument in microns. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_micron(Mono_Enum : integer; Wavelength : double): wordbool;
stdcall;

VB : Private Declare Function ARC_set_Mono_Wavelength_micron Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The wavelength in microns that the instrument is to be moved to.

Result : True if the operation was successful

ARC_get_Mono_Wavelength_absCM , returns the current center wavelength of instrument in absolute wave number. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_absCM(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Wavelength_absCM Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The current wavelength of the instrument in Absolute Wavenumber.

Result : True if the operation was successful

ARC_set_Mono_Wavelength_absCM , sets the center wavelength of the instrument in absolute wavenumbers. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_absCM(Mono_Enum : integer; Wavelength : double): wordbool;
stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_absCM Lib "ARC_Instrument.dll" (ByVal Mono_Enum
As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The wavelength in Absolute Wavenumber that the instrument is to be moved to.

Result : True if the operation was successful

ARC_get_Mono_Wavelength_relCM , returns the current center wavelength of instrument in relative wavenumber based upon the center line value passed to the function in nm. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_relCM(Mono_Enum : integer; Center_nm : double; out Wavelength : double): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Wavelength_relCM Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Center_nm As Double, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Center_nm : The wavelength in nanometers that 0 relative Wavenumber is centered around.

Wavelength : The current wavelength of the instrument in Relative Wavenumber.

Result : True if the operation was successful

ARC_set_Mono_Wavelength_relCM , sets the center wavelength of the instrument in relative wave numbers centered on the wavelength passed in nm. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_relCM(Mono_Enum : integer; Center_nm : double; Wavelength : double): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Wavelength_relCM Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Center_nm As Double, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Center_nm : The wavelength in nanometers that 0 relative Wavenumber is centered around.

Wavelength : The wavelength in relative Wavenumber that the instrument is to be moved to.

Result : True if the operation was successful

ARC_get_Mono_Wavelength_Cutoff_nm , returns in nm the max wavelength achievable by the instrument using the current grating. Note, value might be higher than physically achievable on linear drives using the older SD-748 based controllers.

Delphi : function ARC_get_Mono_Wavelength_Cutoff_nm(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Wavelength_Cutoff_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The maximum center wavelength in nanometers for the current grating. On SpectraPro's this value equals $1400 \text{ nm} * \text{grating density} / 1200 \text{ g/mm}$. On AM/VM products, this value is limited by the sine bar and will vary.

Result : True if the operation was successful

ARC_get_Mono_Wavelength_Min_nm , returns in nm the min wavelength achievable by the instrument using the current grating. Note, value might be lower than physically achievable on linear drives using the older SD-748 based controllers.

Delphi : function ARC_get_Mono_Wavelength_Min_nm(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Wavelength_Min_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength : The minimum center wavelength in nanometers for the current grating. On SpectraPro's it is usually -10 nm, on linear AM/VM instruments this value will vary.

Result : True if the operation was successful

ARC_get_Mono_Turret , returns the current grating turret

Delphi : function ARC_get_Mono_Turret(Mono_Enum : integer; out Turret : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Turret Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Turret As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Turret : Current turret, One plus grating number divided by the number of gratings per turret. This value seldom exceeds three.

Result : True if the operation was successful

ARC_set_Mono_Turret , sets the current grating turret

Delphi : function ARC_set_Mono_Turret(Mono_Enum : integer; Turret : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Turret Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Turret As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Turret : set the current turret. This will change the grating to Turret number minus one times gratings per turret plus the one plus the current grating number minus one mod gratings per turret. The user must insert the correct turret into the instrument when using this function.

Result : True if the operation was successful

ARC_get_Mono_Turret_Gratings , returns the number of gratings per turret

Delphi : function ARC_get_Mono_Turret_Gratings(Mono_Enum : integer; out Gratings_Per_Turret : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Turret_Gratings Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Gratings_Per_Turret As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Gratings_Per_Turret : The number of gratings that can be placed on a single turret. This number can be one, two or three depending on the monochromator model.

Result : True if the operation was successful

ARC_get_Mono_Turret_Max , return the number of turrets installed

Delphi : function ARC_get_Mono_Turret_Max(Mono_Enum : integer; out Max_Turret_Number : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Turret_Max Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Max_Turret As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Max_Turret : The number of turrets installed in the monochromator. This number can be one, two, or three depending on how the instrument was configured. Some custom AM/VM monochromators with 'snapin' gratings have had as many as 9 turrets, this is not normal.

Result : True if the operation was successful

ARC_get_Mono_Grating , returns the current grating

Delphi : function ARC_get_Mono_Grating(Mono_Enum : integer; out Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The current grating. This assumes the correct turret has been inserted in the instrument.

Result : True if the operation was successful

ARC_set_Mono_Grating , sets the current grating

Delphi : function ARC_set_Mono_Grating(Mono_Enum : integer; Grating : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Grating Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : Set the current grating. This assumes the correct turret is inserted in the instrument and grating selected is a valid grating. This function will change to current turret in the firmware, but needs user interaction to install the correct turret.

Result : True if the operation was successful

ARC_get_Mono_Grating_Blaze , returns the blaze of a given grating

Delphi : function ARC_get_Mono_Grating_Blaze(Mono_Enum : integer; Grating : integer; out Blaze_Var :
olevariant): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Grating_Blaze Lib "ARC_Instrument.dll" (ByVal Mono_Enum As
Long, ByVal Grating As Long, Blaze_Var As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : Which grating we are requesting the information about. Validates the request by calling

ARC_get_Mono_Grating_Installed.

Blaze_Var : A variant string of the grating blaze.

Result : True if the grating requested is installed

ARC_get_Mono_Grating_Density , returns the groove density (grooves per millimeter) of a given grating

Delphi : function ARC_get_Mono_Grating_Density(Mono_Enum : integer; Grating : integer; out Groove_MM : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Grating_Density Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, Groove_MM As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : Which grating we are requesting the information about. Validates the request by calling ARC_get_Mono_Grating_Installed.

Groove_MM : The groove density of the grating in grooves per millimeter. For a mirror, this function will return 1200 grooves per MM.

Result : True if the grating requested is installed

ARC_get_Mono_Grating_Installed , returns if a grating is installed

Delphi : function ARC_get_Mono_Grating_Installed(Mono_Enum : integer; Grating : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating_Installed Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : Which grating we are requesting the information about.

Result : True if the grating requested is installed

ARC_get_Mono_Grating_Max , returns the max grating position installed (this is usually the number of gratings installed)

Delphi : function ARC_get_Mono_Grating_Max(Mono_Enum : integer; out Max_Grating_Number : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Grating_Max Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long , Max_Grating_Number As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Max_Grating_Number : The number of gratings installed in the monochromator. This number can range from one to nine. Some custom systems have had as many as twelve, but this is not the normal case.

Result : True if the monochromator is valid

ARC_get_Mono_Diverter_Valid , returns if a motorized diverter position is valid for an instrument

Delphi : function ARC_get_Mono_Diverter_Valid(Mono_Enum : integer; Diverter_Num : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Diverter_Valid Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Diverter_Num As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Diverter_Num : The diverter to be queried.

Mirror Position 1 ... motorized entrance diverter mirror

Mirror Position 2 ... motorized exit diverter mirror

Mirror Position 3 ... motorized entrance diverter on a double slave unit

Mirror Position 4 ... motorized exit diverter on a double slave unit

Result : True if the diverter mirror is motorized.

ARC_get_Mono_Diverter_Pos , returns the slit the mirror is pointing to

Delphi : function ARC_get_Mono_Diverter_Pos(Mono_Enum : integer; Diverter_Num : integer; out Diverter_Pos : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Diverter_Pos Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Diverter_Num As Long, Diverter_Pos As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Diverter_Num : The diverter to be queried.

Mirror Position 1 ... motorized entrance diverter mirror

Mirror Position 2 ... motorized exit diverter mirror

Mirror Position 3 ... motorized entrance diverter on a double slave unit

Mirror Position 4 ... motorized exit diverter on a double slave unit

Diverter_Pos : Port the diverter mirror is currently pointing at.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Result : True if the diverter is motorized and the operation is successful

ARC_set_Mono_Diverter_Pos , sets the port that the unit is to point to

Delphi : function ARC_set_Mono_Diverter_Pos(Mono_Enum : integer; Diverter_Num : integer; Diverter_Pos : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Diverter_Pos Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Diverter_Num As Long, ByVal Diverter_Pos As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Diverter_Num : The diverter to be modified.

Mirror Position 1 ... motorized entrance diverter mirror

Mirror Position 2 ... motorized exit diverter mirror

Mirror Position 3 ... motorized entrance diverter on a double slave unit

Mirror Position 4 ... motorized exit diverter on a double slave unit

Diverter_Pos : Port the diverter mirror is to be set to. Not all ports are valid for all diverters.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Result : True if the diverter is motorized and the operation is successful

ARC_get_Mono_Diverter_Pos_Var , returns a string describing the port the mirror is pointing to

Delphi : function ARC_get_Mono_Diverter_Pos_Var(Mono_Enum : integer; Diverter_Num : integer; out Diverter_Pos : olevariant): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Diverter_Pos_Var Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Diverter_Num As Long, Diverter_Pos As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Diverter_Num : The diverter to be queried.

Mirror Position 1 ... motorized entrance diverter mirror

Mirror Position 2 ... motorized exit diverter mirror

Mirror Position 3 ... motorized entrance diverter on a double slave unit

Mirror Position 4 ... motorized exit diverter on a double slave unit

Diverter_Pos : Returns as a variant a text string describing which port the diverter is pointing at.

Result : True if the diverter is motorized and the operation is successful

ARC_get_Mono_Slit_Type , returns the slit type for a given slit

Delphi : function ARC_get_Mono_Slit_Type(Mono_Enum : integer; Slit_Pos : integer; out Slit_Type : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Slit_Type Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long, Slit_Type As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Slit_Pos : The slit to be addressed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Slit_Type : The type of slit attached

Slit Type 0 ... No slit, older instruments that do will only return a slit type of zero.

Slit Type 1 ... Manual Slit

Slit Type 2 ... Fixed Slit Width

Slit Type 3 ... Focal Plane adapter

Slit Type 4 ... Continuous Motor

Slit Type 5 ... Fiber Optic

Slit Type 6 ... Index able Slit

Slit Type 9 ... Special Continuous Motor (placed here for completeness, only one in existence, you do not have it, trust me)

Result : True if the operation was successful

ARC_get_Mono_Slit_Type_Var , returns a string descriptor of a given slit

Delphi : function ARC_get_Mono_Slit_Type_Var(Mono_Enum : integer; Slit_Pos : integer; out Slit_Type :
olevariant): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Slit_Type_Var Lib "ARC_Instrument.dll" (ByVal Mono_Enum As
Long, ByVal Slit_Pos As Long, Slit_Type As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Slit_Pos : The slit to be addressed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Slit_Type : : Returns as a variant a text string describing the slit.

Result : True if the operation was successful

ARC_get_Mono_Slit_Width , returns the slit width of a motorized slit (Types 4,6, and 9)

Delphi : function ARC_get_Mono_Slit_Width(Mono_Enum : integer; Slit_Pos : integer; out Slit_Width : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Slit_Width Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long, Slit_Width As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Slit_Pos : The slit to be addressed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Slit_Width : Returns the width of the slit if the slit is motorized.

Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful.

ARC_set_Mono_Slit_Width , sets the slit width of a motorized slit (Types 4,6, and 9)

Delphi : function ARC_set_Mono_Slit_Width(Mono_Enum : integer; Slit_Pos : integer; Slit_Width : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Slit_Width Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long, ByVal Slit_Width As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Slit_Pos : The slit to be addressed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Slit_Width : The width to which the slit is to be set to. Note valid ranges are 10 to 3000 microns in one-micron increments for normal continuous motor slits (type 4 and 9), 10 to 12000 microns in five-micron increments for large continuous motor slits (type 4) and 25,50,100,250,500,1000 microns for indexable slits (Type 6).

Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful.

ARC_Mono_Slit_Home , homes a motorized slit (Types 4,6, and 9)

Delphi : function ARC_Mono_Slit_Home(Mono_Enum : integer; Slit_Pos : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Mono_Slit_Home Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Slit_Pos : The slit to be homed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful.

ARC_Mono_Slit_Name_Var , returns a text descriptor of a slit position. The same terminology is used in all Acton Research Monochromators and Spectrographs so this function is used independent of the Mono_Enum variable.

Delphi : function ARC_Mono_Slit_Name_Var(Slit_Num : integer; out Slit_Name_Var : olevariant) : wordbool;
stdcall;

VB : Private Declare Function ARC_Mono_Slit_Name_Var Lib "ARC_Instrument.dll" (ByVal Slit_Num As Long,
Slit_Name_Var As Variant) As Integer

Slit_Pos : The slit to be addressed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

Slit_Name_Var : : Returns as a variant a text string describing the slit port location.

Result : True if the slit number is a valid slit

ARC_Calc_Mono_Slit_BandPass , calculates and returns the band pass for a provided slit width

Delphi : function ARC_Calc_Mono_Slit_BandPass(Mono_Enum : integer; Slit_Pos : integer; SlitWidth : double; out BandPass : double): wordbool; stdcall;

VB : Private Declare Function ARC_Calc_Mono_Slit_BandPass Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos as Long, ByVal SlitWidth as Double, BandPass as Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Slit_Pos : The slit to be homed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

SlitWidth : The slit width that the band pass is being calculated for.

BandPass : The calculated band pass for a slit returned in nm units.

Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful.

ARC_get_Mono_Slit_BandPass , returns the band pass of a motorized slit (Types 4,6, and 9)

function ARC_get_Mono_Slit_BandPass(Mono_Enum : integer; Slit_Pos : integer; out BandPass : double): wordbool;
stdcall;

VB : Private Declare Function ARC_get_Mono_Slit_BandPass Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos as Long, BandPass as Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Slit_Pos : The slit to be homed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

BandPass : The calculated band pass for a slit returned in nm units.

Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful.

ARC_set_Mono_Slit_BandPass , sets a motorized slit width of a motorized slit (Types 4,6, and 9)

```
function ARC_set_Mono_Slit_BandPass(Mono_Enum : integer; Slit_Pos : integer; BandPass : double): wordbool;  
stdcall;  
VB : Private Declare Function ARC_set_Mono_Slit_BandPass Lib "ARC_Instrument.dll" (ByVal Mono_Enum As  
Long, ByVal Slit_Pos as Long, ByVal BandPass as Double) As Integer
```

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Slit_Pos : The slit to be homed.

Slit Port 1 ... Side entrance slit

Slit Port 2 ... Front entrance slit

Slit Port 3 ... Front exit slit

Slit Port 4 ... Side exit slit

Slit Port 5 ... Side entrance slit on a double slave unit

Slit Port 6 ... Front entrance slit on a double slave unit

Slit Port 7 ... Front exit slit on a double slave unit

Slit Port 8 ... Side exit slit on a double slave unit

BandPass : The band pass (in nm units) that the slit will be set to. Changes the slit width.

Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful.

ARC_get_Mono_Filter_Present , returns if the instrument has an integrated filter wheel . Note, is a new option and not available on most instruments. All filter functions call this function before proceeding.

Delphi : function ARC_get_Mono_Filter_Present(Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Filter_Present Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if an integrated filter wheel is present on the monochromator.

ARC_get_Mono_Filter_Position , returns the current filter position

Delphi : function ARC_get_Mono_Filter_Position(Mono_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Filter_Position Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Position As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Position : Returns the current filter wheel position.

Result : True if the filter wheel is present and the operation was successful

ARC_set_Mono_Filter_Position , sets the current filter position

Delphi : function ARC_set_Mono_Filter_Position(Mono_Enum : integer; Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Filter_Position Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Position As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Position : Position the filter is to be set to.

Result : True if the filter wheel is present, position is not outside of ARC_get_Mono_Filter_Min_Pos / ARC_get_Mono_Filter_Max_Pos and the operation was successful

ARC_get_Mono_Filter_Min_Pos , returns the min. filter position

Delphi : function ARC_get_Mono_Filter_Min_Pos(Mono_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Filter_Min_Pos Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Position As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Position : Returns the minimum position possible with the filter wheel.

Result : True if the filter wheel is present and the operation was successful

ARC_get_Mono_Filter_Max_Pos , returns the max. filter position

Delphi : function ARC_get_Mono_Filter_Max_Pos(Mono_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Filter_Max_Pos Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Position As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Position : Returns the maximum position possible with the filter wheel.

Result : True if the filter wheel is present and the operation was successful

ARC_Mono_Filter_Home , homes the filter wheel

Delphi : function ARC_Mono_Filter_Home(Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Mono_Filter_Home Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long)
As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the filter wheel is present, the home operation succeeded, and the operation was successful.

ARC_get_Mono_Shutter_Valid, returns if the instrument has an integrated shutter

Delphi : function ARC_get_Mono_Shutter_Valid (Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Shutter_Valid Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the shutter is present.

ARC_get_Mono_Shutter_Open, returns if the integrated shutter is open

Delphi : function ARC_get_Mono_Shutter_Open (Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Shutter_Open Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the shutter is open.

ARC_set_Mono_Shutter_Open, sets the integrated shutter position to open

Delphi : function ARC_set_Mono_Shutter_Open (Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Shutter_Open Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the shutter has been opened.

ARC_set_Mono_Shutter_Closed, sets the integrated shutter position to closed

Delphi : function ARC_set_Mono_Shutter_Closed (Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Shutter_Closed Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the shutter has been closed.

ARC_get_Mono_Int_Led_On , returns if the interrupter LED is on

Delphi : function ARC_get_Mono_Int_Led_On(Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Int_Led_On Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the operation was successful and the LED is on.

Note : The LED is inside of the monochromator chamber and if turned on, should be turned off with

ARC_set_Mono_Int_LED.

ARC_set_Mono_Int_Led , turns on and off the interrupter LED

Delphi : function ARC_set_Mono_Int_Led(Mono_Enum : integer; Led_State : wordbool): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Int_Led Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Led_State As Integer) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Led_State : Indicates if the LED is **on** or **off**.

0 – False, The LED is off

1 – True, The LED is on

Result : True if the operation was successful.

Note : The LED is inside of the monochromator chamber and if turned on, should be turned off with **ARC_set_Mono_Int_LED**.

ARC_get_Mono_Motor_Int , read the motor gear interrupter

Delphi : function ARC_get_Mono_Motor_Int(Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Motor_Int Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the operation was successful and the interrupter is open.

0 – False, The interrupter is closed, or the LED is Off, or the operation failed

1 – True, The LED is on

ARC_get_Mono_Wheel_Int , read the wheel gear interrupter

Delphi : function ARC_get_Mono_Wheel_Int(Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Wheel_Int Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the operation was successful and the interrupter is open.

0 – False, The interrupter is closed, or the LED is Off, or the operation failed

1 – True, The LED is on

ARC_Mono_Move_Steps , move the grating a set number of steps

Delphi : function ARC_Mono_Move_Steps(Mono_Enum : integer; Num_Steps : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Move_Steps Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Num_Steps As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Num_Steps : Number of steps to move the wavelength drive.

Result : True if the operation was successful.

ARC_get_Mono_Sine_Drive, returns if the gearing system has sine instead of a linear drive system.

Delphi : function ARC_get_Mono_Sine_Drive(Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Sine_Drive Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the gearing is sine based verses linear gearing.

ARC_get_Mono_Init_Grating , returns the initial grating (on instrument reboot)

Delphi : function ARC_get_Mono_Init_Grating(Mono_Enum : integer; out Init_Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Init_Grating Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Init_Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Init_Grating : The power-up grating number.

Result : True if the operation was successful.

ARC_set_Mono_Init_Grating , sets the initial grating (on instrument reboot)

Delphi : function ARC_set_Mono_Init_Grating(Mono_Enum : integer; Init_Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Init_Grating Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Init_Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Init_Grating : The power-up grating number.

Result : True if the operation was successful.

ARC_get_Mono_Init_Wave_nm , returns the initial wavelength (on instrument reboot)

Delphi : function ARC_get_Mono_Init_Wave_nm(Mono_Enum : integer; out Init_Wave : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Init_Wave_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Init_Wave As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Init_Wave : The power-up wavelength in nm.

Result : True if the operation was successful.

ARC_set_Mono_Init_Wave_nm , sets the initial wavelength (on instrument reboot)

Delphi : function ARC_set_Mono_Init_Wave_nm(Mono_Enum : integer; Init_Wave : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Init_Wave_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Init_Wave As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Init_Wave : The power-up wavelength in nm.

Result : True if the operation was successful.

ARC_get_Mono_Init_ScanRate_nm , returns the initial wavelength scan rate (on instrument reboot)

Delphi : function ARC_get_Mono_Init_ScanRate_nm(Mono_Enum : integer; out Init_ScanRate : double): wordbool;
stdcall;

VB : Private Declare Function ARC_get_Mono_Init_ScanRate_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum
As Long, Init_ScanRate As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Init_ScanRate : The power-up scan rate in nm / minute.

Result : True if the operation was successful.

ARC_set_Mono_Init_ScanRate_nm , sets the initial wavelength scan rate (on instrument reboot)

Delphi : function ARC_set_Mono_Init_ScanRate_nm(Mono_Enum : integer; Init_ScanRate : double): wordbool;
stdcall;

VB : Private Declare Function ARC_set_Mono_Init_ScanRate_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Init_ScanRate As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Init_ScanRate : The power-up scan rate in nm / minute.

Result : True if the operation was successful.

ARC_get_Mono_Grating_Offset , returns the offset if a grating

Delphi : function ARC_get_Mono_Grating_Offset(Mono_Enum : integer; Grating : integer; out Offset : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Grating_Offset Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, Offset As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The number of the grating that is to be addressed.

Offset : The grating offset. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.

Result : True if the operation was successful.

ARC_get_Mono_Grating_Gadjust , returns the GAdjust of a grating

Delphi : function ARC_get_Mono_Grating_Gadjust(Mono_Enum : integer; Grating : integer; out Gadjust : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_Grating_Gadjust Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, Gadjust As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The number of the grating that is to be addressed.

GAdjust : The grating GAdjust. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.

Result : True if the operation was successful.

ARC_set_Mono_Grating_Offset , sets the grating offset

Delphi : function ARC_set_Mono_Grating_Offset(Mono_Enum : integer; Grating : integer; Offset : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Grating_Offset Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Offset As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The number of the grating that is to be addressed.

Offset : The grating offset. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.

Result : True if the operation was successful.

Note : Should only be used in conjunction with **ARC_Mono_Grating_Calc_Offset**.

ARC_set_Mono_Grating_Gadjust , sets the grating GAdjust

Delphi : function ARC_set_Mono_Grating_Gadjust(Mono_Enum : integer; Grating : integer; Gadjust : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Mono_Grating_Gadjust Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Gadjust As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The number of the grating that is to be addressed.

Gadjust : The grating GAdjust. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.

Result : True if the operation was successful.

Note : Should only be used in conjunction with **ARC_Mono_Grating_Calc_GAdjust**.

ARC_Mono_Grating_Calc_Offset , calculate a new grating offset

Delphi : function ARC_Mono_Grating_Calc_Offset(Mono_Enum : integer; Grating : integer; Wave : double; RefWave : double; out newOffset : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Mono_Grating_Calc_Offset Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Wave As Double, ByVal RefWave As Double, newOffset As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The number of the grating that is to be addressed.

Wave : The wavelength in nm on which a peak is currently falling.

RefWave : The wavelength in nm on which the peak should fall.

newOffset : The calculated grating offset. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.

Result : True if the operation was successful.

ARC_Mono_Grating_Calc_Gadjust , calculate a new grating GAdjust

Delphi : function ARC_Mono_Grating_Calc_Gadjust(Mono_Enum : integer; Grating : integer; Wave : double;
RefWave : double; out newGadjust : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Grating_Calc_Gadjust Lib "ARC_Instrument.dll" (ByVal Mono_Enum As
Long, ByVal Grating As Long, ByVal Wave As Double, ByVal RefWave As Double, newGadjust As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The number of the grating that is to be addressed.

Wave : The wavelength in nm on which a peak is currently falling.

RefWave : The wavelength in nm on which the peak should fall.

newGAdjust : The calculated grating GAdjust. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.

Result : True if the operation was successful.

ARC_Mono_Grating_UnInstall , Uninstall a grating

Delphi : function ARC_Mono_Grating_UnInstall(Mono_Enum : integer; Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Grating_UnInstall Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The number of the grating that is to be uninstalled.

Result : True if the operation was successful.

ARC_Mono_Grating_Install , Install a new grating

Delphi : function ARC_Mono_Grating_Install(Mono_Enum : integer; Grating : integer; Density : Integer; Blaze : OleVariant; NMBlaze : boolean; HOLBlaze : boolean; MIRROR : boolean): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Grating_Install Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Density As Long, ByVal Blaze As Variant, ByVal NMBlaze As Long, ByVal HOLBlaze As Long, ByVal MIRROR As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Grating : The number of the grating that is to be addressed.

Density : The groove density in grooves per mm of the grating.

Blaze : The Blaze String.

NMBlaze : If the grating has an nm blaze verses an micron blaze.

0 – False, Ruled grating with a micron blaze

1 – True, Ruled grating with a nm blaze

HOLBlaze : If the grating has a Holographic Blaze.

0 – False, Is not a Holographic Grating

1 – True, Is a Holographic Grating

MIRROR : If the grating position is a mirror.

0 – False, Not a mirror

1 – True, Is a mirror

Result : True if the operation was successful.

ARC_Mono_Reset , Reboot an instrument

Delphi : function ARC_Mono_Reset(Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Mono_Reset Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the operation was successful and the instrument was reset to the power-on state.

ARC_Mono_Restore_Factory_Settings , restore the instrument factory settings (not available of the oldest instruments)

Delphi : function ARC_Mono_Restore_Factory_Settings(Mono_Enum : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Restore_Factory_Settings Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the operation was successful.

Note : It is highly recommended that this function not be invoked.

ARC_Mono_Save_Factory_Settings, this definition is reserved and should never be invoked

Delphi : reserved

VB : reserved

Reserved : reserved

ARC_get_Mono_Scan_Rate_nm_min , return the current wavelength scan rate

Delphi : function ARC_get_Mono_Scan_Rate_nm_min (Mono_Enum : integer; out Scan_Rate : double): wordbool;
stdcall;

VB : Private Declare Function ARC_get_Mono_Scan_Rate_nm_min Lib "ARC_Instrument.dll" (ByVal Mono_Enum
As Long, Scan_Rate As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Scan_Rate : Scan rate in nm per minute.

Result : True if the operation was successful.

ARC_set_Mono_Scan_Rate_nm_min , set the wavelength scan rate

Delphi : function ARC_set_Mono_Scan_Rate_nm_min (Mono_Enum : integer; Scan_Rate : double): wordbool;
stdcall;

VB : Private Declare Function ARC_set_Mono_Scan_Rate_nm_min Lib "ARC_Instrument.dll" (ByVal Mono_Enum
As Long, ByVal Scan_Rate As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Scan_Rate : The rate we are to scan the wavelength in nm/minute.

Result : True if the operation was successful.

ARC_Mono_Start_Scan_To_nm , start a wavelength scan

Delphi : function ARC_Mono_Start_Scan_To_nm (Mono_Enum : integer; Wavelength_nm : double): wordbool;
stdcall;

VB : Private Declare Function ARC_Mono_Start_Scan_To_nm Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, ByVal Wavelength_nm As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Wavelength_nm : Wavelength in nm we are to scan to. Note should not be lower than the current wavelength and should not exceed the current grating wavelength limit.

Result : True if the operation was successful.

ARC_Mono_Scan_Done , check if a scan has completed

Delphi : function ARC_Mono_Scan_Done (Mono_Enum : integer; out Done_Moving : boolean; out

Current_Wavelength_nm : double): wordbool; stdcall;

VB : Private Declare Function ARC_Mono_Scan_Done Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long, Done_Moving As Long, Current_Wavelength_nm As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Done_Moving : Indicate if we are done moving (the scan ended or the grating wavelength limit was reached)

0 – False, we are still moving

1 – True, we are done moving

Current_Wavelength_nm : The wavelength in nm of the grating when this command was issued.

Result : True if the operation was successful.

Note : Once a scan or a jog is completed a **ARC_Mono_Stop_Jog** must be issued once before an accurate final wavelength can be read with **ARC_get_Mono_Wavelength_nm**.

ARC_Mono_Start_Jog , start jogging the wavelength of an instrument

Delphi : function ARC_Mono_Start_Jog (Mono_Enum : integer; Jog_MaxRate : boolean; JogUp : boolean): wordbool;
stdcall;

VB : Use not recommended in the VB environment

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Jog_MaxRate : If we jog at the max scan rate or current scan rate.

0 – Current Scan Rate

1 – Max Scan Rate

JogUp : If we jog forward (increasing nm) or backwards (decreasing nm)

0 – Jog Backwards (decreasing nm)

1 – Jog Forwards (increasing nm)

Result : True if the operation was successful.

ARC_Mono_Stop_Jog , end a wavelength jog

Delphi : function ARC_Mono_Stop_Jog (Mono_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Mono_Stop_Jog Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Result : True if the operation was successful.

Monochromator SP-2xxx and SD-3 Reserved Setup functions

These functions are used internally by Acton Research Corp. during the manufacturing process to calibrate and configure Acton systems. They are documented for completeness only. Acton Research Corp. does not provide support for these functions to the end user. Do not use these functions.

ARC_Mono_Set_Model, reserved
ARC_Mono_Set_Serial, reserved
ARC_Mono_Get_Board_Serial, reserved
ARC_Mono_Set_Board_Serial, reserved
ARC_Mono_Set_Drive_Linear, reserved
ARC_Mono_Set_Drive_Sine, reserved
ARC_Mono_Set_Gratings_Per_Turret , reserved
ARC_Mono_Set_Halfangle, reserved
ARC_Mono_Set_Gear_Ratio, reserved
ARC_Mono_Set_Backlash, reserved
ARC_Mono_Read_Motors, reserved
ARC_Mono_Motor_set_Use, reserved
ARC_Mono_Motor_set_Stop, reserved
ARC_Mono_Motor_set_Speed, reserved
ARC_Mono_Motor_set_StepRev, reserved
ARC_Mono_Motor_set_eoptions, reserved
ARC_Mono_Grating_Install_NoHello, reserved
ARC_Mono_Grating_UnInstall_NoHello, reserved
ARC_Mono_Motor_set_Motor_Pos_Step, reserved
ARC_Mono_Motor_set_Motor_Min_Pos, reserved
ARC_Mono_Motor_set_Motor_Max_Pos, reserved
ARC_Mono_Motor_set_SlitClamp, reserved
ARC_Mono_Motor_set_SFreq, reserved
ARC_Mono_Motor_set_Microns_to_Steps, reserved
ARC_Mono_Motor_set_Steps_to_Microns, reserved
ARC_Mono_Motor_set_SlitHome, reserved
ARC_Mono_Motor_get_Slit_Hz, reserved
ARC_Mono_Motor_get_Motor_Chan, reserved
ARC_Mono_Motor_get_Motor_StepStop, reserved
ARC_Mono_Motor_get_Motor_Accel, reserved
ARC_Mono_Motor_get_Motor_lraf, reserved
ARC_Mono_Motor_get_Motor_hraf, reserved
ARC_Mono_Motor_get_Motor_mper, reserved
ARC_Mono_Motor_get_Motor_App, reserved
ARC_Mono_Motor_get_Motor_Index_Min_Pos , reserved
ARC_Mono_Motor_get_Motor_Index_Max_Pos, reserved
ARC_Mono_Motor_get_Motor_Speed, reserved
ARC_Mono_Motor_get_Motor_Step_Offset, reserved
ARC_Mono_Motor_get_Motor_Step_Rev, reserved
ARC_Mono_Motor_get_Motor_Step_Pos, reserved
ARC_Mono_Set_Focallength, reserved
ARC_Mono_Set_DetectorAngle, reserved
ARC_Mono_Clear_Mono_Setup, reserved

ARC_Open_Filter , function to open a Filter Wheel

Delphi : function ARC_Open_Filter(Enum_Num : integer; out Filter_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Open_Filter Lib "ARC_Instrument.dll" (ByVal Enum_Num As Long,
Filter_Enum As Long) As Integer

Enum_Num : Number in the enumeration list to open.

Filter_Enum : The handle by which to address the Filter Wheel in all future calls. Can be validated with
ARC_Valid_Filter_Enum.

Result : True, if it successfully opened the device.

ARC_Open_Filter_Port , function to open a Filter Wheel on a specific COM Port (ie. COM1, COM2, ect...)

Delphi : function ARC_Open_Filter_Port(Com_Num : integer; out Filter_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Open_Filter_Port Lib "ARC_Instrument.dll" (ByVal Com_Num As Long, Filter_Enum As Long) As Integer

Com_Num : Com Port (COM1,COM2,ect.) to open.

Filter_Enum : The handle by which to address the Filter Wheel in all future calls. Can be validated with ARC_Valid_Filter_Enum.

Result : True, if it successfully opened the device.

ARC_Valid_Filter_Enum , function returns if a Filter Wheel enumeration is both valid and the instrument is open. All functions call this function to verify that the requested action is valid.

Delphi : function ARC_Valid_Filter_Enum(Filter_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Valid_Filter_Enum Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Result : True if Filter_Enum is a valid Filter Wheel that is open.

ARC_get_Filter_preOpen_Model , returns the model string of an instrument not yet opened. Note, ARC_Search_For_Filter needs to be called prior to this call. In the case of multiple Filter Wheel/Spectrographs attached, it allows the user to sort, which instruments are to be opened before opening them.

Delphi : function ARC_get_Filter_preOpen_Model(Enum_Num : integer; out Model_Var : olevariant) : wordbool; stdcall;

VB : Private Declare Function ARC_get_Filter_preOpen_Model Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long, Model_Var As Variant) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Model_Var : The model string of the unopened Filter Wheel.

Result : True if the operation was successful

ARC_Send_CMD_To_Filter , this is an internal function that provides direct communications with the instrument. Usage of this function should be avoided.

Delphi : function ARC_Send_CMD_To_Filter(Filter_Enum : integer; SendCMD : olevariant; out RCV_Var : olevariant; ms_Timeout : integer) : wordbool; stdcall;

VB : Private Declare Function ARC_Send_CMD_To_Filter Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long, ByVal SendCMD As Variant, RCV_Var As Variant, ByVal ms_Timeout As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

SendCMD : The text command to be sent to the instrument. Note, line termination is handled by the DLL.

RCV_Var : The reply from the instrument to the given command.

Ms_Timeout : How long the DLL should wait in milliseconds for an acknowledgement before returning

Result : True if the instrument did not reject the command or timeout.

ARC_get_Filter_Model , returns the model string from the instrument

Delphi : function ARC_get_Filter_Model(Filter_Enum : integer;out Model_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Model Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long,
Model_str As Variant) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Model_Var : Returns the model string of the instrument.

Result : True if the operation was successful

ARC_get_Filter_Serial , returns the serial number from the instrument. Note, older filter controllers did not electronically store the serial number. In the case of the older controllers Serial_Var will not contain any text.

Delphi : function ARC_get_Filter_Serial(Filter_Enum : integer;out Serial_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Serial Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long, Serial_str As Variant) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Serial_Var : Returns the serial number of the instrument as a string.

Result : True if the operation was successful

ARC_get_Filter_Present , returns if the instrument has a filter wheel . Note if ARC_Valid_Filter_Enum returns true, this function will also return true.

Delphi : function ARC_get_Filter_Present(Filter_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Filter_Present Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Result : True if an integrated filter wheel is present on the Filter Wheel.

ARC_get_Filter_Position , returns the current filter position

Delphi : function ARC_get_Filter_Position(Filter_Enum : integer; out Position : integer): wordbool; stdcall;

VB : Private Declare Function ARC_get_Filter_Position Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long, Position As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Position : Returns the current filter wheel position.

Result : True if the filter wheel is present and the operation was successful

ARC_set_Filter_Position , sets the current filter position

Delphi : function ARC_set_Filter_Position(Filter_Enum : integer; Position : integer): wordbool; stdcall;

VB : Private Declare Function ARC_set_Filter_Position Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long, ByVal Position As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Position : Position the filter is to be set to.

Result : True if the filter wheel is present, position is not outside of ARC_get_Filter_Min_Pos / ARC_get_Filter_Max_Pos and the operation was successful

ARC_get_Filter_Min_Pos , returns the min. filter position

Delphi : function ARC_get_Filter_Min_Pos(Filter_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Min_Pos Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long, Position As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Position : Returns the minimum position possible with the filter wheel.

Result : True if the filter wheel is present and the operation was successful

ARC_get_Filter_Max_Pos , returns the max. filter position

Delphi : function ARC_get_Filter_Max_Pos(Filter_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Max_Pos Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long, Position As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Position : Returns the maximum position possible with the filter wheel.

Result : True if the filter wheel is present and the operation was successful

ARC_Filter_Home , homes the filter wheel

Delphi : function ARC_Filter_Home(Filter_Enum : integer): wordbool; stdcall;

VB : Private Declare Function ARC_Filter_Home Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Result : True if the filter wheel is present, the home operation succeeded, and the operation was successful.

ARC_get_Filter_preOpen_Serial_int32 , returns the serial number (as an integer) of a filterwheel not yet open.

Delphi : function ARC_get_Filter_preOpen_Serial_int32(Filter_Num : integer; out Serial_Int32 : integer) : wordbool;
stdcall;

VB : Private Declare Function ARC_get_Filter_preOpen_Serial_int32 Lib "ARC_Instrument.dll" (ByVal Filter_Num
As Long , Serial_Int32 As Long) As Integer

Filter_Num : The index as defined by ARC_Search_For_Inst for the instrument addressed.

Serial_Int32 : Returns the serial number as an integer.

Result : True if Filter_Num refers to a valid filterwheel.

ARC_get_Mono_preOpen_Serial_int32 , returns the serial number (as an integer) of a monochromator not yet open.

Delphi : function ARC_get_Mono_preOpen_Serial_int32(Mono_Num : integer; out Serial_Int32 : integer) : wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_preOpen_Serial_int32 Lib "ARC_Instrument.dll" (ByVal Mono_Num As Long , Serial_Int32 As Long) As Integer

Mono_Num : The index as defined by ARC_Search_For_Inst for the instrument addressed.

Serial_Int32 : Returns the serial number as an integer.

Result : True if Mono_Num refers to a valid monochromator.

ARC_get_ReadOut_preOpen_Serial_int32 , returns the serial number (as an integer) of an instrument not yet open.

Delphi : function ARC_get_ReadOut_preOpen_Serial_int32(Readout_Enum : integer; out Serial_Int32 : integer) : wordbool; stdcall;

VB : Private Declare Function ARC_get_ReadOut_preOpen_Serial_int32 int32 Lib "ARC_Instrument.dll" (ByVal Inst_Num As Long , Serial_Int32 As Long) As Integer

Inst_Num : The index as defined by ARC_Search_For_Inst for the instrument addressed.

Serial_Int32 : Returns the serial number as an integer.

Result : True if Num refers to a valid readout.

ARC_get_Filter_Serial_int32 , return the serial number of a filterwheel as an integer

Delphi : function ARC_get_Filter_Serial_int32(Filter_Enum : integer) : integer; stdcall;

VB : Private Declare Function ARC_get_Filter_Serial_int32(Lib "ARC_Instrument.dll" (ByVal Filter_Enum As Long) As Long)

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.

Result : The serial number as an integer of the filterwheel. This function will break if the serial number contains non numeric characters.

ARC_get_Mono_Serial_int32 , return the serial number of a monochromator as an integer

Delphi : function ARC_get_Mono_Serial_int32(Mono_Enum : integer) : integer; stdcall;

VB : Private Declare Function ARC_get_Mono_Serial_int32(Lib "ARC_Instrument.dll" (ByVal Mono_Enum As Long) As Long)

Filter_Enum : The handle defined in ARC_Open_Mono by which the Mono is to be addressed.

Result : The serial number as an integer of the monochromator. This function will break if the serial number contains non numeric characters.

ARC_get_ReadOut_Serial_int32 , return the serial number of a readout as an integer

Delphi : function ARC_get_ReadOut_Serial_int32(ReadOut_Enum : integer) : integer; stdcall;

VB : Private Declare Function ARC_get_ReadOut_Serial_int32(Lib "ARC_Instrument.dll" (ByVal ReadOut_Enum As Long) As Long)

Filter_Enum : The handle defined in ARC_Open_ReadOut by which the ReadOut is to be addressed.

Result : The serial number as an integer of the ReadOut. This function will break if the serial number contains non numeric characters.

ARC_get_Mono_preOpen_Model_int32 , returns the model type of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call.

Delphi : function ARC_get_Mono_preOpen_Model_Int32(Enum_Num : integer; out Model_Int32 : integer) : wordbool; stdcall;

VB : Private Declare Function ARC_get_Mono_preOpen_Model_int32(ByVal Enum_Num As Long , Model_Int32 As Long) As Integer

Enum_Num : The index as defined by ARC_Search_For_Inst for the instrument addressed.

Model_Int32 : The model type as a integer

Model Type -1 ... Unknown, not supported

Model Type 0 ... Unknown, not supported

Model Type 1 ... GIMS (reserved for, none currently supported by dll)

Model Type 2 ... SP-150 (superceded by SP-2-150 (SPv6x) in 2003)

Model Type 3 ... SPx00i (generic gen2 board) (superceded by SPv6x in 2003)

Model Type 4 ... SPx004i (generic SD-2) (superceded by SD-3 in 2003)

Model Type 5 ... SP-300i (superceded by SP-2-300i (SPv6x) in 2003)

Model Type 6 ... SP-500i (superceded by SP-2-500i (SPv6x) in 2003)

Model Type 7 ... SP-750i (superceded by SP-2-750i (SPv6x) in 2003)

Model Type 8 ... INS-300 (serial number is < 42) (superceded by SPv6x based INS-300 in 2003)

Model Type 9 ... VM-502i (VM-502 with SD-2) (superceded by SD-3 based VM-502 in 2003)

Model Type 10 ... VM-504i (VM-504 with SD-2) (superceded by SD-3 based VM-504 in 2003)

Model Type 11 ... VM-503 (VM-503 with SD-2) (superceded by SD-3 based VM-503 in 2003)

Model Type 12 ... VM-505 (VM-505 with SD-2) (superceded by SD-3 based VM-505 in 2003)

Model Type 13 ... VM-506 (VM-506 with SD-2) (superceded by SD-3 based VM-506 in 2003)

Model Type 14 ... VM-507 (VM-507 with SD-2) (superceded by SD-3 based VM-507 in 2003)

Model Type 15 ... VM-510 (VM-510 with SD-2) (superceded by SD-3 based VM-510 in 2003)

Model Type 16 ... VM-511 (VM-511 with SD-2) (superceded by SD-3 based VM-511 in 2003)

Model Type 17 ... VM-512 (VM-512 with SD-2) (superceded by SD-3 based VM-512 in 2003)

Model Type 18 ... VM-521 (VM-521 with SD-2) (superceded by SD-3 based VM-521 in 2003)

Model Type 19 ... VM-523 (VM-523 with SD-2) (superceded by SD-3 based VM-523 in 2003)

Model Type 20 ... SP-275 (superceded by SP-300i in 1996)

Model Type 21 ... SP-500 (SP-500 with SD-748) (superceded by SP-500i in 1996)

Model Type 22 ... SP-750 (SP-750 with SD-748) (superceded by SP-750i in 1998)

Model Type 23 ... VM-502 (VM-502 with SD-748) (superceded by VM-502i in 1999)

Model Type 24 ... VM-504 (VM-504 with SD-748) (superceded by VM-504i in 1999)

Model Type 25 ... SD-748 (superceded by SD-2 in 1999)

Model Type 26 ... INS-150

Model Type 27 ... SPv6x (All SpectraPro's produced since 2003,

Includes: SP-2-150, SP-2-300i, SP-2-500i, SP-2-750i, SD-3,

InSight, MS-2-150i, MS-2-300i, INS-300 serial numbers > 42)

Model Type 28 ... NCL (superceded by SpectraHub in 2003)

Model Type 29 ... Filter Controller

Model Type 30 ... Reserved for Sample Chambers

Model Type 31 ... Reserved (custom, not supported by dll)

Model Type 32 ... Reserved (custom, not supported by dll)

Model Type 33 ... SpectraHub

Result : True if the operation was successful

ARC_get_ReadOut_preOpen_Model_int32, returns the model type of an instrument not yet opened. Note, **ARC_Search_For_Inst** needs to be called prior to this call.

Delphi : function **ARC_get_ReadOut_preOpen_Model_int32** (Enum_Num : integer; out Model_Int32 : integer) : wordbool; stdcall;

VB : Private Declare Function **ARC_get_ReadOut_preOpen_Model_int32** (ByVal Enum_Num As Long , Model_Int32 As Long) As Integer

Enum_Num : The index as defined by **ARC_Search_For_Inst** for the instrument addressed.

Model_Int32 : The model type as an integer

Model Type -1 ... Unknown, not supported

Model Type 0 ... Unknown, not supported

Model Type 1 ... GIMS (reserved for, none currently supported by dll)

Model Type 2 ... SP-150 (superceded by SP-2-150 (SPv6x) in 2003)

Model Type 3 ... SPx00i (generic gen2 board) (superceded by SPv6x in 2003)

Model Type 4 ... SPx004i (generic SD-2) (superceded by SD-3 in 2003)

Model Type 5 ... SP-300i (superceded by SP-2-300i (SPv6x) in 2003)

Model Type 6 ... SP-500i (superceded by SP-2-500i (SPv6x) in 2003)

Model Type 7 ... SP-750i (superceded by SP-2-750i (SPv6x) in 2003)

Model Type 8 ... INS-300 (serial number is < 42) (superceded by SPv6x based INS-300 in 2003)

Model Type 9 ... VM-502i (VM-502 with SD-2) (superceded by SD-3 based VM-502 in 2003)

Model Type 10 ... VM-504i (VM-504 with SD-2) (superceded by SD-3 based VM-504 in 2003)

Model Type 11 ... VM-503 (VM-503 with SD-2) (superceded by SD-3 based VM-503 in 2003)

Model Type 12 ... VM-505 (VM-505 with SD-2) (superceded by SD-3 based VM-505 in 2003)

Model Type 13 ... VM-506 (VM-506 with SD-2) (superceded by SD-3 based VM-506 in 2003)

Model Type 14 ... VM-507 (VM-507 with SD-2) (superceded by SD-3 based VM-507 in 2003)

Model Type 15 ... VM-510 (VM-510 with SD-2) (superceded by SD-3 based VM-510 in 2003)

Model Type 16 ... VM-511 (VM-511 with SD-2) (superceded by SD-3 based VM-511 in 2003)

Model Type 17 ... VM-512 (VM-512 with SD-2) (superceded by SD-3 based VM-512 in 2003)

Model Type 18 ... VM-521 (VM-521 with SD-2) (superceded by SD-3 based VM-521 in 2003)

Model Type 19 ... VM-523 (VM-523 with SD-2) (superceded by SD-3 based VM-523 in 2003)

Model Type 20 ... SP-275 (superceded by SP-300i in 1996)

Model Type 21 ... SP-500 (SP-500 with SD-748) (superceded by SP-500i in 1996)

Model Type 22 ... SP-750 (SP-750 with SD-748) (superceded by SP-750i in 1998)

Model Type 23 ... VM-502 (VM-502 with SD-748) (superceded by VM-502i in 1999)

Model Type 24 ... VM-504 (VM-504 with SD-748) (superceded by VM-504i in 1999)

Model Type 25 ... SD-748 (superceded by SD-2 in 1999)

Model Type 26 ... INS-150

Model Type 27 ... SPv6x (All SpectraPro's produced since 2003,

Includes: SP-2-150, SP-2-300i, SP-2-500i, SP-2-750i, SD-3,

InSight, MS-2-150i, MS-2-300i, INS-300 serial numbers > 42)

Model Type 28 ... NCL (superceded by SpectraHub in 2003)

Model Type 29 ... Filter Controller

Model Type 30 ... Reserved for Sample Chambers

Model Type 31 ... Reserved (custom, not supported by dll)

Model Type 32 ... Reserved (custom, not supported by dll)

Model Type 33 ... SpectraHub

Result : True if the operation was successful

ARC_get_Filter_preOpen_Model_int32, returns the model type of an instrument not yet opened. Note, **ARC_Search_For_Inst** needs to be called prior to this call.

Delphi : function **ARC_get_Filter_preOpen_Model_int32** (Enum_Num : integer; out Model_Int32 : integer) : wordbool; stdcall;

VB : Private Declare Function **ARC_get_Filter_preOpen_Model_int32** (ByVal Enum_Num As Long , Model_Int32 As Long) As Integer

Enum_Num : The index as defined by **ARC_Search_For_Inst** for the instrument addressed.

Model_Int32 : The model type as an integer

Model Type -1 ... Unknown, not supported

Model Type 0 ... Unknown, not supported

Model Type 1 ... GIMS (reserved for, none currently supported by dll)

Model Type 2 ... SP-150 (superceded by SP-2-150 (SPv6x) in 2003)

Model Type 3 ... SPx00i (generic gen2 board) (superceded by SPv6x in 2003)

Model Type 4 ... SPx004i (generic SD-2) (superceded by SD-3 in 2003)

Model Type 5 ... SP-300i (superceded by SP-2-300i (SPv6x) in 2003)

Model Type 6 ... SP-500i (superceded by SP-2-500i (SPv6x) in 2003)

Model Type 7 ... SP-750i (superceded by SP-2-750i (SPv6x) in 2003)

Model Type 8 ... INS-300 (serial number is < 42) (superceded by SPv6x based INS-300 in 2003)

Model Type 9 ... VM-502i (VM-502 with SD-2) (superceded by SD-3 based VM-502 in 2003)

Model Type 10 ... VM-504i (VM-504 with SD-2) (superceded by SD-3 based VM-504 in 2003)

Model Type 11 ... VM-503 (VM-503 with SD-2) (superceded by SD-3 based VM-503 in 2003)

Model Type 12 ... VM-505 (VM-505 with SD-2) (superceded by SD-3 based VM-505 in 2003)

Model Type 13 ... VM-506 (VM-506 with SD-2) (superceded by SD-3 based VM-506 in 2003)

Model Type 14 ... VM-507 (VM-507 with SD-2) (superceded by SD-3 based VM-507 in 2003)

Model Type 15 ... VM-510 (VM-510 with SD-2) (superceded by SD-3 based VM-510 in 2003)

Model Type 16 ... VM-511 (VM-511 with SD-2) (superceded by SD-3 based VM-511 in 2003)

Model Type 17 ... VM-512 (VM-512 with SD-2) (superceded by SD-3 based VM-512 in 2003)

Model Type 18 ... VM-521 (VM-521 with SD-2) (superceded by SD-3 based VM-521 in 2003)

Model Type 19 ... VM-523 (VM-523 with SD-2) (superceded by SD-3 based VM-523 in 2003)

Model Type 20 ... SP-275 (superceded by SP-300i in 1996)

Model Type 21 ... SP-500 (SP-500 with SD-748) (superceded by SP-500i in 1996)

Model Type 22 ... SP-750 (SP-750 with SD-748) (superceded by SP-750i in 1998)

Model Type 23 ... VM-502 (VM-502 with SD-748) (superceded by VM-502i in 1999)

Model Type 24 ... VM-504 (VM-504 with SD-748) (superceded by VM-504i in 1999)

Model Type 25 ... SD-748 (superceded by SD-2 in 1999)

Model Type 26 ... INS-150

Model Type 27 ... SPv6x (All SpectraPro's produced since 2003,

Includes: SP-2-150, SP-2-300i, SP-2-500i, SP-2-750i, SD-3,

InSight, MS-2-150i, MS-2-300i, INS-300 serial numbers > 42)

Model Type 28 ... NCL (superceded by SpectraHub in 2003)

Model Type 29 ... Filter Controller

Model Type 30 ... Reserved for Sample Chambers

Model Type 31 ... Reserved (custom, not supported by dll)

Model Type 32 ... Reserved (custom, not supported by dll)

Model Type 33 ... SpectraHub

Result : True if the operation was successful